

C# / .NET Learning Roadmap

Topic Notes — Section 2: Object-Oriented Programming in C#

Topics: 2.1 through 2.17

Date: 29 June 2026

Section 2 — Object-Oriented Programming in C#

This section covers OOP in C# — the principles and language features that structure every .NET application. Topics build on the C# fundamentals from Section 1.

2.1. Classes and Objects

Builds on: 1.4 (reference types — classes are reference types), 1.11 (methods — classes contain them), 1.16 (structs vs classes — you already know the contrast), 1.19 (scope — class fields have class-level scope).

What is a class and why does it exist?

A class is a blueprint. An object is what you create from that blueprint.

Without classes, you'd have loose variables and methods floating around with no relationship to each other. A class groups related data (fields/properties) and behaviour (methods) into a single named unit — the fundamental building block of object-oriented programming.

```
// Blueprint:
public class Order
{
    public int Id { get; set; }
    public string CustomerName { get; set; }
    public decimal Amount { get; set; }

    public bool IsHighValue() => Amount > 10000;
}

// Objects – instances of the blueprint:
var order1 = new Order { Id = 1, CustomerName = "Sathish", Amount = 15000 };
var order2 = new Order { Id = 2, CustomerName = "Priya", Amount = 500 };

Console.WriteLine(order1.IsHighValue()); // true
Console.WriteLine(order2.IsHighValue()); // false
```

order1 and order2 are independent objects. Same blueprint, different data.

Anatomy of a class

```
public class BankAccount
{
    // Fields -- raw storage (usually private)
    private decimal _balance;

    // Properties -- controlled access to data
    public string AccountNumber { get; private set; }
    public decimal Balance => _balance; // read-only property

    // Constructor -- runs when object is created with new
    public BankAccount(string accountNumber, decimal initialBalance)
    {
        AccountNumber = accountNumber;
        _balance = initialBalance;
    }
}
```

```

    }

    // Methods -- behaviour
    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("Amount must be positive");
        _balance += amount;
    }

    public bool Withdraw(decimal amount)
    {
        if (amount > _balance) return false;
        _balance -= amount;
        return true;
    }

    public override string ToString()
        => $"Account {AccountNumber}: {_balance:C}";
}

```

Fields vs Properties — the practical distinction

Fields are raw variables declared directly in the class. By convention, private and prefixed with `_`:

```

private decimal _balance;
private readonly ILogger _logger;

```

Properties expose data with controlled access. The most common forms:

```

// Auto-property -- compiler generates backing field:
public string Name { get; set; }

// Read-only auto-property:
public string Id { get; }

// Private setter -- only this class can set it:
public string Status { get; private set; }

// Computed property -- no backing field, calculated on demand:
public decimal TotalWithTax => Amount * 1.18m;

// Init-only -- can be set in object initialiser but not after:
public string Reference { get; init; }

```

In practice: fields are private, properties are public. Never expose raw fields publicly — always use properties so you can add validation or change implementation later without breaking callers.

Object initialiser syntax

```

var order = new Order
{
    Id = 1,
    CustomerName = "Sathish",
    Amount = 999.99m
};

```

Only works for settable properties ({ get; set; } or { get; init; }). Cannot call methods or set private fields through this syntax.

The this keyword

```
public class Order
{
    public string Status { get; private set; }

    public void SetStatus(string status)
    {
        this.Status = status;
    }

    private string _name;
    public Order(string name)
    {
        _name = name;
    }
}
```

In practice, the `_` prefix convention for fields means you rarely need this for disambiguation.

Access modifiers on class members

Modifier	Accessible from
-----+-----	
public	Anywhere
private	This class only
protected	This class and subclasses
internal	This assembly (project) only
private protected	This class and subclasses in same assembly
protected internal	This assembly OR subclasses anywhere

Static class members

```
public class Order
{
    private static int _nextId = 1;

    public int Id { get; }

    public Order()
    {
        Id = _nextId++;
    }

    public static Order CreateEmpty()
        => new Order();
}

var a = new Order();           // Id = 1
var b = new Order();           // Id = 2
var c = Order.CreateEmpty();   // Id = 3
```

Where classes appear in real .NET code

Entities — map to database tables:

```
public class User
{
    public int Id { get; set; }
    public string Email { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

DTOs — carry data between layers:

```
public class CreateOrderDto
{
    public string CustomerName { get; set; }
    public decimal Amount { get; set; }
}
```

Services — contain business logic:

```
public class OrderService
{
    private readonly IOrderRepository _repo;

    public OrderService(IOrderRepository repo) => _repo = repo;

    public async Task<Order> CreateAsync(CreateOrderDto dto) { ... }
}
```

Realistic scenario — a complete small class

```
public class PasswordValidator
{
    private readonly int _minLength;
    private readonly bool _requireSpecialChar;

    public PasswordValidator(int minLength = 8, bool requireSpecialChar = true)
    {
        _minLength = minLength;
        _requireSpecialChar = requireSpecialChar;
    }

    public (bool isValid, string error) Validate(string password)
    {
        if (string.IsNullOrEmpty(password))
            return (false, "Password is required");

        if (password.Length < _minLength)
            return (false, $"Password must be at least {_minLength} characters");

        if (_requireSpecialChar && !password.Any(c => !char.IsLetterOrDigit(c)))
            return (false, "Password must contain a special character");

        return (true, string.Empty);
    }
}

var validator = new PasswordValidator(minLength: 10, requireSpecialChar: true);
```

```
var (isValid, error) = validator.Validate("MyPass123!");
if (!isValid) return BadRequest(error);
```

Key Takeaways

- A class is a blueprint defining data (fields/properties) and behaviour (methods)
- An object is an instance created from a class with `new`
- Fields are private raw storage; properties are controlled public access
- Multiple objects from the same class are independent
- Static members belong to the type and are shared; instance members belong to each object

Quick-reference — member types:

Member	Purpose	Typical access
Field	Raw storage	private
Property	Controlled access to data	public
Constructor	Initialise the object	public
Method	Behaviour / operations	public or private
Static	Shared across all instances	public static

.NET-specific rules:

- Properties over public fields — always
- Prefix private fields with `_`
- `{ get; init; }` for immutable-after-creation properties
- override `ToString()` when the class will appear in logs or debug output

Memory aid: Class = blueprint, Object = house built from it. One blueprint, many independent houses.

Debugging Tips

Symptom	Cause	Fix
<code>NullReferenceException</code> on property	Object never initialised	Check null; ensure <code>new</code> called
Property change not reflected	May have copied reference	Verify reference vs new object
Cannot access private member	Accessing private from outside	Expose via public property
Object initialiser won't compile	No setter or private set	Add <code>set</code> , <code>init</code> , or use ctor
Static member gives shared data	Forgot it's shared across reqs	Move to instance or use DI

.NET-specific: In ASP.NET Core, services registered as `Singleton` behave like static — one instance shared across all requests. Use `Scoped` for per-request state.

Self-check — you should now be able to:

- Define a class with private fields, public properties, a constructor, and methods
- Explain the difference between a class (blueprint) and an object (instance)
- Choose between `{ get; set; }`, `{ get; }`, `{ get; private set; }`, and `{ get; init; }` correctly
- Explain what static means on a class member and when to use it
- Identify the four most common class types in a .NET API (Entity, DTO, Service, Controller)

2.2. Constructors (Default, Parameterized, Static, Primary Constructors)

Builds on: 2.1 (classes — constructors are what run when you call new), 1.12 (optional parameters — constructor parameters follow the same rules), 1.18 (readonly — fields set in constructors are the primary readonly pattern).

What are constructors and why do they exist?

Without a constructor, there's no controlled way to ensure an object starts in a valid state. You'd create an object and then have to remember to set all the right properties manually — easy to forget, easy to get wrong.

A constructor is a special method that runs automatically when you create an object with new. Its job is to initialise the object into a valid, usable state from the first moment it exists.

Default Constructor

If you don't define any constructor, C# generates a hidden parameterless one that sets all fields to their defaults:

```
public class Order
{
    public int Id { get; set; }
    public string Status { get; set; }
}

// The compiler generates this invisibly:
// public Order() { }

var order = new Order(); // works -- uses generated default constructor
order.Id = 1;
order.Status = "Pending";
```

The moment you define any constructor yourself, the compiler no longer generates the default one. If you still want a parameterless constructor alongside your own, you must write it explicitly.

Parameterised Constructor

```
public class Order
{
    public int Id { get; }
    public string CustomerName { get; }
    public decimal Amount { get; }
    public string Status { get; private set; }

    public Order(int id, string customerName, decimal amount)
    {
        if (string.IsNullOrEmpty(customerName))
            throw new ArgumentException("Customer name is required");
        if (amount <= 0)
            throw new ArgumentException("Amount must be positive");

        Id = id;
    }
}
```

```

        CustomerName = customerName;
        Amount = amount;
        Status = "Pending";
    }
}

var order = new Order(1, "Sathish", 999.99m);
// order is guaranteed valid from the first line

```

Constructor Overloading

```

public class EmailMessage
{
    public string To { get; }
    public string Subject { get; }
    public string Body { get; }
    public bool IsHtml { get; }

    // Minimal constructor:
    public EmailMessage(string to, string subject)
        : this(to, subject, string.Empty, false) // calls the full constructor
    {
    }

    // Full constructor:
    public EmailMessage(string to, string subject, string body, bool isHtml)
    {
        To = to;
        Subject = subject;
        Body = body;
        IsHtml = isHtml;
    }
}

var simple = new EmailMessage("s@zoho.com", "Hello");
var full = new EmailMessage("s@zoho.com", "Hello", "<b>Hi</b>", true);

```

The `: this(...)` syntax is constructor chaining — one constructor calls another on the same class. This avoids duplicating initialisation logic across multiple constructors. The chained constructor runs first, then the calling constructor's body runs.

Constructor Chaining with `: base()`

```

public class Entity
{
    public int Id { get; }
    public DateTime CreatedAt { get; }

    public Entity(int id)
    {
        Id = id;
        CreatedAt = DateTime.UtcNow;
    }
}

public class Order : Entity

```



```

{
    public string CustomerName { get; }

    public Order(int id, string customerName)
        : base(id)    // calls Entity(int id) first
    {
        CustomerName = customerName;
    }
}

```

base(id) runs the Entity constructor before Order's body. The base class is always initialised first.

Static Constructor

```

public class AppConfig
{
    public static readonly string Environment;
    public static readonly bool IsProduction;

    // No parameters, no access modifier, runs once automatically:
    static AppConfig()
    {
        Environment = System.Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT")
            ?? "Development";
        IsProduction = Environment == "Production";
    }
}

Console.WriteLine(AppConfig.Environment);    // "Development" or "Production"
Console.WriteLine(AppConfig.IsProduction);    // false or true

```

Rules for static constructors: no access modifier, no parameters, runs at most once per app lifetime, cannot be called manually, runs before the first use of the class.

Primary Constructors — C# 12

```

// Traditional:
public class OrderService
{
    private readonly IOrderRepository _repo;
    private readonly ILogger<OrderService> _logger;

    public OrderService(IOrderRepository repo, ILogger<OrderService> logger)
    {
        _repo = repo;
        _logger = logger;
    }
}

// With primary constructor -- C# 12:
public class OrderService(IOrderRepository repo, ILogger<OrderService> logger)
{
    public async Task<Order?> GetByIdAsync(int id)
    {
        logger.LogInformation("Getting order {Id}", id);
        return await repo.GetByIdAsync(id);
    }
}

```

```
    }
}
```

The parameters (repo, logger) are available throughout the entire class body without declaring fields. Primary constructor parameters are not automatically readonly. Capture them explicitly for robustness:

```
public class OrderService(IOrderRepository repo, ILogger<OrderService> logger)
{
    private readonly IOrderRepository _repo = repo;
    private readonly ILogger<OrderService> _logger = logger;

    public async Task<Order?> GetByIdAsync(int id)
    {
        _logger.LogInformation("Getting order {Id}", id);
        return await _repo.GetByIdAsync(id);
    }
}
```

Primary constructors on records are different — they generate public properties automatically:

```
public record Order(int Id, string CustomerName, decimal Amount);
```

Object Initialisers vs Constructors

```
// Object initialiser -- good for DTOs and simple data containers:
var dto = new CreateOrderDto { CustomerName = "Sathish", Amount = 999.99m };
```

```
// Constructor -- good when the object needs validation or computed initial state:
var order = new Order(1, "Sathish", 999.99m);
```

For domain objects and entities, prefer constructors. For DTOs and configuration objects, object initialisers are fine.

Realistic scenario — DI constructor in ASP.NET Core

```
[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly IOrderService _orderService;
    private readonly ILogger<OrdersController> _logger;

    public OrdersController(IOrderService orderService, ILogger<OrdersController> logger)
    {
        _orderService = orderService;
        _logger = logger;
    }

    [HttpGet("{id}")]
    public async Task<IActionResult> GetById(int id)
    {
        _logger.LogInformation("Fetching order {Id}", id);
        var order = await _orderService.GetByIdAsync(id);
        return order is null ? NotFound() : Ok(order);
    }
}
```

ASP.NET Core's DI container reads the constructor parameters, resolves the registered services, and injects them. You never call this constructor yourself — the framework does.

Key Takeaways

- A constructor runs automatically when new is called — it initialises the object into a valid state
- If you define any constructor, the compiler stops generating the default parameterless one
- Constructor chaining (: this(...)) lets one constructor call another to avoid duplicating logic
- Static constructors run once before first use, have no parameters and no access modifier
- Primary constructors (C# 12) declare parameters on the class declaration — capture into readonly fields for robustness

Constructor type	When it runs	Parameters	Access modifier
Default (generated)	new ClassName()	None	public
Parameterised	new ClassName(args)	Yes	Any
Chained (: this)	Inside another ctor	Varies	N/A
Base (: base)	Inside derived ctor	Varies	N/A
Static	Before first class use	None	None
Primary (C# 12)	new ClassName(args)	On decl	Any

.NET-specific rules:

- Always validate constructor parameters — throw ArgumentException or ArgumentNullException for invalid input
- Use ArgumentNullException.ThrowIfNull(param) (.NET 6+) for clean null checks
- In ASP.NET Core, the DI container resolves constructor parameters — register all dependencies in Program.cs
- EF Core requires a parameterless constructor on entity classes (can be private or protected)

Memory aid: Constructor = the object's birth certificate. It must be filled in correctly before the object enters the world.

Debugging Tips

Symptom	Cause	Fix
No parameterless constructor	Added parameterised ctor	Add public ClassName() {}
DI throws Unable to resolve	Type not registered in DI	Add registration in Program.
Object reference not set	Required property not set	Use constructor that require
Static constructor runs multiple	It doesn't -- check instance	Static ctors run exactly once
Primary ctor param is null	DI failed or null passed	Validate with ThrowIfNull

Self-check — you should now be able to:

- Write a class with a parameterised constructor that validates its inputs
- Explain what happens when you define a constructor and why the default disappears
- Use constructor chaining (: this(...)) to avoid duplicated initialisation logic
- Explain when a static constructor runs and what it's used for

- Write the standard DI constructor pattern used in every ASP.NET Core service and controller
- Explain the difference between primary constructors and traditional constructors in C# 12

2.3. Properties (get/set, Auto-Properties, init-only)

Builds on: 2.1 (classes — properties are class members), 2.2 (constructors — init-only properties are set there), 1.18 (readonly — init-only is the property equivalent).

What are properties and why do they exist?

A public field gives direct, uncontrolled access to data:

```
public class Order
{
    public decimal Amount;    // anyone can set this to -99999
}
```

```
var order = new Order();
order.Amount = -500;    // no validation possible -- bad
```

Properties solve this by wrapping data access in getter and setter methods — but with cleaner syntax than Java-style GetAmount()/SetAmount().

```
public class Order
{
    private decimal _amount;

    public decimal Amount
    {
        get { return _amount; }
        set
        {
            if (value <= 0) throw new ArgumentException("Amount must be positive");
            _amount = value;
        }
    }
}
```

```
order.Amount = -500;    // throws ArgumentException -- protected
order.Amount = 999;    // fine
```

Auto-Properties — the shorthand

```
// Full manual property:
private string _name;
public string Name
{
    get { return _name; }
    set { _name = value; }
}
```

```
// Auto-property -- identical behaviour, compiler generates the field:
```

```
public string Name { get; set; }
```

Use auto-properties when you don't need custom logic in the getter or setter. Add the backing field manually only when you do.

Property access modifier variants

```
public string Status { get; set; }           // Anyone can read and write
public string Status { get; private set; }   // Externally read-only
public string Status { get; protected set; } // Subclasses can write
public int Id { get; }                       // Set only in constructor
public string Reference { get; init; }       // Set in initialiser only
```

init-only properties — C# 9

```
public class Order
{
    public int Id { get; init; }
    public string CustomerName { get; init; }
    public decimal Amount { get; init; }
}

// Object initialiser -- allowed:
var order = new Order { Id = 1, CustomerName = "Sathish", Amount = 999.99m };

// After construction -- not allowed:
order.Id = 2;    // compile error -- init-only
```

{ get; } vs { get; init; }: { get; } requires a constructor. { get; init; } works with object initialisers too — more flexible for DTOs and value objects that don't need constructors.

Computed properties — no backing field

```
public class Order
{
    public decimal Subtotal { get; set; }
    public decimal TaxRate { get; set; }

    public decimal Tax => Subtotal * TaxRate;
    public decimal Total => Subtotal + Tax;
    public bool IsHighValue => Total > 10000;
}

var order = new Order { Subtotal = 1000m, TaxRate = 0.18m };
Console.WriteLine(order.Tax);    // 180
Console.WriteLine(order.Total);  // 1180
```

Full property with custom logic

```
public class User
{
    private string _email;

    public string Email
    {
        get => _email;
    }
}
```

```

        set
        {
            if (string.IsNullOrEmpty(value))
                throw new ArgumentException("Email is required");
            if (!value.Contains('@'))
                throw new ArgumentException("Invalid email format");
            _email = value.ToLower().Trim();    // normalise on store
        }
    }
}

```

Required properties — C# 11

```

public class CreateOrderDto
{
    public required string CustomerName { get; set; }
    public required decimal Amount { get; set; }
    public string? Notes { get; set; }    // optional
}

var dto = new CreateOrderDto();    // compile error -- required properties not set

var dto = new CreateOrderDto { CustomerName = "Sathish", Amount = 999.99m };    // fine

```

Properties in EF Core

```

public class Product
{
    public int Id { get; set; }                // primary key
    public string Name { get; set; }           // NOT NULL column
    public string? Description { get; set; }    // nullable column
    public decimal Price { get; set; }         // NOT NULL column
    public bool IsActive { get; set; } = true; // default value
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

```

Realistic scenario — DTO and entity using different property styles

```

// Entity -- controlled mutation, immutable ID:
public class Order
{
    public int Id { get; }
    public string CustomerName { get; private set; }
    public decimal Amount { get; private set; }
    public string Status { get; private set; } = "Pending";
    public DateTime CreatedAt { get; } = DateTime.UtcNow;

    public Order(int id, string customerName, decimal amount)
    {
        Id = id; CustomerName = customerName; Amount = amount;
    }

    public void UpdateAmount(decimal newAmount)
    {
        if (newAmount <= 0) throw new ArgumentException("Invalid amount");
    }
}

```

```

        Amount = newAmount;
    }
}

// DTO -- open for model binding, init-only for immutability:
public class CreateOrderDto
{
    public required string CustomerName { get; init; }
    public required decimal Amount { get; init; }
    public string? PromoCode { get; init; }
}

```

Key Takeaways

- Properties control access to data — they look like fields but behave like methods
- Auto-properties ({ get; set; }) are the default — add a backing field only when you need custom logic
- { get; } — read-only, set only in constructor
- { get; init; } — read-only after construction, settable in object initialisers
- { get; private set; } — externally readable, only the class can write
- Computed properties (=> expression) derive from other data, recalculate on each access
- required (C# 11) enforces that properties are set in object initialisers

Syntax	Can read	Can write	When writable
-----+-----+-----+-----			
{ get; set; }	Anyone	Anyone	Anytime
{ get; private set; }	Anyone	This class	Anytime (in class)
{ get; protected set; }	Anyone	Class+sub	Anytime (hierarchy)
{ get; }	Anyone	This class	Constructor only
{ get; init; }	Anyone	Anyone	Initialisation only
=> expression	Anyone	N/A	Read-only, computed

.NET-specific rules:

- EF Core entities need { get; set; } — EF must be able to set values
- DTOs use { get; init; } or { get; set; } depending on whether you want post-construction mutation
- Never expose public fields — always properties
- Use required on DTOs instead of constructors when you want object initialiser syntax with enforced completeness

Memory aid: Property = a security guard for your data. The guard decides who gets in and checks credentials before letting anyone change anything.

Debugging Tips

Symptom	Cause	Fix
-----+-----+-----		
EF Core skips property in results	No setter or private set	Add set; or use HasField
init property can't be set	Only during initialisation	Use private set if needed
Model binding doesn't populate	No setter	Add { get; set; } or init
Computed property returns stale	Caching result in variable	Access property each time

required property error | Missing in object initialiser | Set all required properties

.NET-specific: JSON deserialization (System.Text.Json) requires either a set or init accessor to populate properties. { get; } only properties will be silently skipped during deserialization.

Self-check — you should now be able to:

- Choose the right property syntax for a given scenario from the six variants in the table
- Explain why public fields should never be used and what properties give you instead
- Write a property with custom validation logic in the setter
- Distinguish between { get; } and { get; init; } and when each is appropriate
- Explain why EF Core entities need { get; set; } and why DTOs can use { get; init; }
- Write a computed property and explain when it recalculates

2.4. Access Modifiers (public, private, protected, internal, etc.)

Builds on: 2.1 (classes — access modifiers appear on every class member), 2.3 (properties — you've already used private set and protected set), 1.2 (project/solution structure — internal is scoped to a project/assembly).

What are access modifiers and why do they exist?

Without access modifiers, every piece of code in your entire application could read and modify every field, method, and property of every class. Access modifiers enforce encapsulation — the principle that a class controls what it exposes and hides its implementation details.

Two things they solve: Safety — prevent external code from putting your objects in invalid states. Flexibility — you can change internal implementation without breaking callers, because callers only depend on what's public.

The Six Access Modifiers

```
public           -- accessible from anywhere
private          -- accessible only within this class
protected        -- accessible within this class and any subclass
internal         -- accessible within this assembly (project) only
protected internal -- accessible within this assembly OR any subclass
private protected -- accessible within this class and subclasses in same assembly
```

Default access modifiers

```
class Order { }           // internal -- only this project can use it
public class Order { }    // public -- any project can use it

public class Order
{
    int _id;                // private -- only this class
    void Process() { }      // private -- only this class
}
```


Rule of thumb: omitting a modifier on a type gives internal; omitting it on a member gives private. Always be explicit — don't rely on defaults.

public — accessible from anywhere

```
public class OrderService
{
    public Order GetById(int id) { ... }
    public async Task<Order> CreateAsync(...) { ... }
}
```

Use public for: API controllers and their action methods, service interfaces and their public methods, DTOs and entity classes consumed by other layers, extension methods. Everything your API consumers need to call must be public.

private — this class only

```
public class OrderService
{
    private readonly IOrderRepository _repo;
    private readonly ILogger _logger;

    public Order GetById(int id)
    {
        LogAccess(id);
        return _repo.FindById(id);
    }

    private void LogAccess(int id)
        => _logger.LogInformation("Accessing order {Id}", id);
}
```

Use private for: all injected dependency fields, internal helper methods, backing fields for properties, implementation details. Default for class members — start with private and open up only when needed.

internal — this assembly (project) only

```
// In MyApp.Infrastructure project:
internal class SqlOrderRepository : IOrderRepository
{
    // The SQL implementation is internal
}

// In MyApp.API project (different .csproj):
var repo = new SqlOrderRepository(); // compile error -- internal to Infrastructure
IOrderRepository repo = ...;         // use the interface instead
```

Use internal for: concrete implementations accessed through interfaces, helper classes used only within a single project, infrastructure details. In Clean Architecture, infrastructure implementations are typically internal.

protected — this class and subclasses

```
public abstract class BaseController : ControllerBase
```

```

{
    protected ILogger Logger { get; }
    protected string CurrentUserId => User.FindFirst("uid")?.Value;

    protected BaseController(ILogger logger) { Logger = logger; }

    protected IActionResult HandleNotFound(string resource)
        => NotFound(new { error = $"{resource} not found" });
}

public class OrdersController : BaseController
{
    [HttpGet("{id}")]
    public IActionResult GetById(int id)
    {
        var order = _repo.FindById(id);
        if (order == null) return HandleNotFound("Order");
        Logger.LogInformation("Order {Id} retrieved", id);
        return Ok(order);
    }
}

```

The principle — expose minimum, hide maximum

```

public class OrderService
{
    private readonly IOrderRepository _repo;
    private readonly IEmailService _emailService;

    public async Task<Order> CreateAsync(CreateOrderDto dto) // public -- API surface
    {
        ValidateDto(dto);
        var order = BuildOrder(dto);
        await _repo.SaveAsync(order);
        await NotifyCustomerAsync(order);
        return order;
    }

    private void ValidateDto(CreateOrderDto dto) { ... }
    private Order BuildOrder(CreateOrderDto dto) { ... }
    private async Task NotifyCustomerAsync(Order order) { ... }
}

```

InternalsVisibleTo — sharing internals with test projects

```
[assembly: InternalsVisibleTo("MyApp.Tests")]
```

Now MyApp.Tests can see internal members of MyApp. This is standard practice — internal helpers remain invisible to real consumers but testable.

Key Takeaways

- Access modifiers enforce encapsulation — controlling what your class exposes

- Start everything private. Open up to internal when another class in the same project needs it. Open up to public only when it forms part of the intended API surface
- public — anyone anywhere; private — only this class; internal — only this project; protected — this class and subclasses

Modifier	Same class	Same assembly	Subclass	Anyone
private	Yes	No	No	No
private protected	Yes	Yes(sub only)	No	No
protected	Yes	No	Yes	No
internal	Yes	Yes	No	No
protected internal	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

.NET-specific rules:

- Default for types: internal; Default for members: private
- ASP.NET Core controllers and action methods must be public
- EF Core can use private setters on entity properties — it uses reflection
- Use [assembly: InternalsVisibleTo("YourTests")] to test internal members
- Interface members are implicitly public — no modifier needed or allowed

Memory aid: Access modifiers are like doors in a building. private = your office. internal = your floor. protected = your floor plus branches of your department. public = the lobby — anyone walks in.

Debugging Tips

Symptom	Cause	Fix
inaccessible due to protection	Accessing private/internal	Expose via public method
Test can't access internal class	Separate assembly	Add InternalsVisibleTo
Controller returns 404 all routes	Action not public	Make action method public
EF Core doesn't populate property	No setter	Add private set;
Interface impl compile error	Access modifier on member	Remove modifier

Self-check — you should now be able to:

- Name all six access modifiers and describe what each allows
- Explain the defaults when no modifier is written for a type and for a member
- Apply the 'start private, open up only when needed' principle to a new class
- Explain why internal is appropriate for concrete infrastructure implementations
- Use [assembly: InternalsVisibleTo] to expose internals to a test project

2.5. Encapsulation

Builds on: 2.1 (classes — encapsulation is the core principle behind why classes exist), 2.3 (properties — the primary mechanism of encapsulation in C#), 2.4 (access modifiers — the enforcement tool for encapsulation).

What is encapsulation and why does it exist?

Encapsulation is the principle of bundling data and the behaviour that operates on that data together, while hiding internal implementation details from the outside world.

It solves a fundamental software problem: when external code can directly manipulate an object's internal state, that object can be put into invalid, inconsistent, or unexpected states — and the object has no way to prevent it.

```
// No encapsulation -- anything goes:
public class BankAccount
{
    public decimal Balance;
    public bool IsActive;
}

var account = new BankAccount();
account.Balance = -99999;    // invalid -- no protection
account.IsActive = false;   // bypasses any deactivation logic
```

With encapsulation, the object controls its own state:

```
public class BankAccount
{
    private decimal _balance;
    private bool _isActive = true;

    public decimal Balance => _balance;
    public bool IsActive => _isActive;

    public void Deposit(decimal amount)
    {
        if (!_isActive) throw new InvalidOperationException("Account is closed");
        if (amount <= 0) throw new ArgumentException("Amount must be positive");
        _balance += amount;
    }

    public void Withdraw(decimal amount)
    {
        if (!_isActive) throw new InvalidOperationException("Account is closed");
        if (amount <= 0) throw new ArgumentException("Amount must be positive");
        if (amount > _balance) throw new InvalidOperationException("Insufficient funds");
        _balance -= amount;
    }

    public void Close() { _isActive = false; }
}

account.Withdraw(10000);    // throws -- balance protected
account.Balance = 999999;  // compile error -- read-only
```

The three layers of encapsulation in C#

1. Private fields — hide raw data:

```
private decimal _balance;
private string _status;
```

```
private readonly List<Transaction> _transactions = new();
```

2. Public properties with controlled access — expose data safely:

```
public decimal Balance => _balance;  
public string Status { get; private set; }  
public IReadOnlyList<Transaction> Transactions => _transactions.AsReadOnly();
```

3. Public methods as the only way to change state — enforce rules:

```
public void ApplyDiscount(decimal percent)  
{  
    if (percent <= 0 || percent >= 100)  
        throw new ArgumentOutOfRangeException(nameof(percent));  
    _balance *= (1 - percent / 100);  
}
```

Tell, don't ask

A key encapsulation principle. Instead of asking an object for its data and then deciding what to do:

```
// BAD -- asking pattern, breaks encapsulation:  
if (order.Status == "Pending" && order.Amount > 0 && order.CustomerId != null)  
{  
    order.Status = "Processing";    // external code modifying state directly  
}
```

Tell the object to do something and let it enforce its own rules:

```
// GOOD -- telling pattern:  
order.StartProcessing();  
  
// Inside Order:  
public void StartProcessing()  
{  
    if (Status != "Pending")  
        throw new InvalidOperationException($"Cannot process an order with status {Status}");  
    if (Amount <= 0)  
        throw new InvalidOperationException("Cannot process order with zero amount");  
  
    Status = "Processing";  
    ProcessingStartedAt = DateTime.UtcNow;  
}
```

Encapsulation with collections

```
// BAD -- collection is exposed mutably:  
public class Order { public List<OrderItem> Items { get; set; } = new(); }  
  
// GOOD -- collection is protected:  
public class Order  
{  
    private readonly List<OrderItem> _items = new();  
  
    public IReadOnlyList<OrderItem> Items => _items.AsReadOnly();  
    public int ItemCount => _items.Count;  
    public decimal Total => _items.Sum(i => i.Price * i.Quantity);  
}
```

```

public void AddItem(OrderItem item)
{
    if (item == null) throw new ArgumentNullException(nameof(item));
    if (_items.Any(i => i.ProductId == item.ProductId))
        throw new InvalidOperationException("Product already in order");
    _items.Add(item);
}

public void RemoveItem(int productId)
{
    var item = _items.FirstOrDefault(i => i.ProductId == productId);
    if (item == null) throw new InvalidOperationException("Item not found");
    _items.Remove(item);
}
}

```

Realistic scenario — domain model with full encapsulation

```

public class Invoice
{
    private readonly List<InvoiceLineItem> _lineItems = new();
    private InvoiceStatus _status = InvoiceStatus.Draft;

    public int Id { get; }
    public string CustomerEmail { get; }
    public DateTime CreatedAt { get; } = DateTime.UtcNow;
    public DateTime? PaidAt { get; private set; }

    public IReadOnlyList<InvoiceLineItem> LineItems => _lineItems.AsReadOnly();
    public InvoiceStatus Status => _status;
    public decimal Total => _lineItems.Sum(i => i.Amount);
    public bool IsPaid => _status == InvoiceStatus.Paid;

    public Invoice(int id, string customerEmail)
    {
        if (string.IsNullOrEmpty(customerEmail))
            throw new ArgumentException("Email required");
        Id = id;
        CustomerEmail = customerEmail;
    }

    public void AddLineItem(string description, decimal amount)
    {
        if (_status != InvoiceStatus.Draft)
            throw new InvalidOperationException("Cannot modify a non-draft invoice");
        _lineItems.Add(new InvoiceLineItem(description, amount));
    }

    public void Issue()
    {
        if (_status != InvoiceStatus.Draft)
            throw new InvalidOperationException("Only draft invoices can be issued");
        if (!_lineItems.Any())
            throw new InvalidOperationException("Cannot issue an empty invoice");
        _status = InvoiceStatus.Issued;
    }
}

```

```

    }

    public void MarkAsPaid()
    {
        if (_status != InvoiceStatus.Issued)
            throw new InvalidOperationException("Only issued invoices can be paid");
        _status = InvoiceStatus.Paid;
        PaidAt = DateTime.UtcNow;
    }
}

```

Key Takeaways

- Encapsulation = bundling data + behaviour together, hiding internal details
- The object is responsible for keeping itself in a valid state
- Three tools: private fields, controlled properties, public methods as state-change entry points
- Tell, don't ask — tell objects to do things rather than reading their state and acting on it externally
- Expose collections as `ReadOnlyList` or `ReadOnlyCollection` — never as `List`
- Every public member is a contract — every private member is yours to change freely

Question	Correctly encapsulated answer
Are fields public?	No -- always private
Can external code put object invalid?	No
Does class enforce its own rules?	Yes -- in methods
Are collections exposed as mutable?	No -- <code>ReadOnlyList<T></code>
Can implementation change without..?	Yes

.NET-specific rules:

- Use `ReadOnlyList` or `ReadOnlyCollection` for exposed collections
- `AsReadOnly()` wraps a `List` without copying it
- EF Core can work with private setters and private backing fields
- In DDD, encapsulation like this is called a rich domain model

Memory aid: Encapsulation is like a vending machine. You press buttons (public methods), you get the result (return values). You never reach inside to grab the mechanism directly.

Debugging Tips

Symptom	Cause	Fix
Object ends up in invalid state	Public fields/settable properties	Make fields private, add
<code>ReadOnlyList</code> throws on Add	Caller casting to <code>List<T></code>	Intentional -- use add m
EF can't load related entities	<code>ReadOnlyList</code> without field config	Configure <code>HasField</code> in EF
Business rule violated	Properties set directly	Remove public set
Unit test can't access state	Private fields hidden	Test through public meth

Self-check — you should now be able to:

- Explain what encapsulation is and the problem it solves in one sentence
- Identify an encapsulation violation in a code snippet and fix it

- Apply the 'tell, don't ask' principle to replace direct property access with method calls
- Expose a collection safely using IReadOnlyList and AsReadOnly()
- Explain why hiding implementation details gives you freedom to change internals later

2.6. Inheritance

Builds on: 2.1 (classes), 2.2 (constructors — base constructors run first), 2.4 (access modifiers — protected exists specifically for inheritance), 2.5 (encapsulation — inheritance can break it if misused).

What is inheritance and why does it exist?

Inheritance lets a class (the derived class) acquire the members of another class (the base class), extending or specialising its behaviour without rewriting it.

The problem it solves: when multiple classes share the same data and behaviour, you'd otherwise duplicate that code across all of them.

```
// Shared behaviour in base class:
public class Payment
{
    public decimal Amount { get; }
    public DateTime ProcessedAt { get; private set; }

    public Payment(decimal amount) { Amount = amount; }

    public void Validate() { if (Amount <= 0) throw new InvalidOperationException(); }
    public void LogPayment() => Console.WriteLine($"Payment of {Amount:C}");
}

// Derived classes extend with specific behaviour:
public class CreditCardPayment : Payment
{
    public string CardNumber { get; }
    public CreditCardPayment(decimal amount, string cardNumber) : base(amount)
        => CardNumber = cardNumber;
}

public class BankTransferPayment : Payment
{
    public string AccountNumber { get; }
    public BankTransferPayment(decimal amount, string accountNumber) : base(amount)
        => AccountNumber = accountNumber;
}
```

virtual and override — customising inherited behaviour

```
public class Payment
{
    public decimal Amount { get; }
    public Payment(decimal amount) => Amount = amount;
    // virtual -- derived classes CAN override this:
```



```

        public virtual string GetDescription() => $"Payment of {Amount:C}";

        // Not virtual -- derived classes CANNOT override this:
        public void LogPayment() => Console.WriteLine(GetDescription());
    }

    public class CreditCardPayment : Payment
    {
        public string LastFour { get; }
        public CreditCardPayment(decimal amount, string lastFour) : base(amount)
            => LastFour = lastFour;

        // override -- replaces the base implementation:
        public override string GetDescription()
            => $"Credit card payment of {Amount:C} (card ending {LastFour})";
    }

    Payment p2 = new CreditCardPayment(100m, "4242");
    Console.WriteLine(p2.GetDescription());
    // "Credit card payment of $100.00 (card ending 4242)"

```

base keyword — calling the parent's implementation

```

    public class CreditCardPayment : Payment
    {
        public override string GetDescription()
        {
            var baseDescription = base.GetDescription();
            return $"{baseDescription} via credit card ending {LastFour}";
        }
    }

```

sealed — preventing further inheritance

```

    public sealed class AdminUser : User
    {
        // No one can inherit from AdminUser
    }

    public class CreditCardPayment : Payment
    {
        public sealed override string GetDescription()
            => $"Credit card payment of {Amount:C}";
    }

```

Inheritance in ASP.NET Core

```

    // Custom exception hierarchy:
    public class AppException : Exception
    {
        public int StatusCode { get; }
        public AppException(string message, int statusCode = 400) : base(message)
            => StatusCode = statusCode;
    }

    public class NotFoundException : AppException

```

```

{
    public NotFoundException(string resource)
        : base($"{resource} was not found", statusCode: 404) { }
}

public class UnauthorizedException : AppException
{
    public UnauthorizedException()
        : base("Unauthorised access", statusCode: 401) { }
}

```

Realistic scenario — Entity base class

```

public abstract class BaseEntity
{
    public int Id { get; private set; }
    public DateTime CreatedAt { get; private set; } = DateTime.UtcNow;
    public DateTime? UpdatedAt { get; private set; }
    public string CreatedBy { get; private set; }

    protected BaseEntity() { }
    protected BaseEntity(string createdBy) { CreatedBy = createdBy; }

    public void SetUpdated() { UpdatedAt = DateTime.UtcNow; }
}

public class Order : BaseEntity
{
    public string CustomerName { get; private set; }
    public decimal Amount { get; private set; }

    protected Order() { }
    public Order(string customerName, decimal amount, string createdBy)
        : base(createdBy) { CustomerName = customerName; Amount = amount; }
}

```

Key Takeaways

- Inheritance lets a derived class acquire members of a base class — is-a relationship
- C# supports single inheritance only — one direct base class
- virtual + override enable polymorphic behaviour
- base.Method() calls the parent's implementation from within an override
- sealed prevents further subclassing or overriding
- Every class ultimately inherits from System.Object
- Prefer composition over inheritance for behaviour reuse — use inheritance for genuine is-a hierarchies

Keyword	Where	Meaning
: BaseClass	Class declaration	Inherit from BaseClass
: base(args)	Constructor	Call base class constructor
virtual	Base class method	Derived classes may override
override	Derived class	Replaces base implementation

sealed	Class or override	Prevents further subclassing
base.Method()	Inside override	Call the base class version
abstract	Class or method	Must be overridden (section 2.8)

Memory aid: Inheritance = genetic traits. A derived class inherits its parent's DNA (members) and can express them differently (override). But you only have one biological parent (single inheritance).

Debugging Tips

Symptom	Cause	Fix
Base method runs instead of ovrd	Method not marked virtual	Add virtual to base
override keyword compile error	Base method isn't virtual	Mark base method virtual
EF throws on entity materialise	Entity lacks parameterless ctor	Add protected EntityName() {
Ctor in derived doesn't compile	No matching base ctor	Add ctor to base or adjust
sealed class being subclassed	Inheriting from sealed class	Remove sealed or use composi

Self-check — you should now be able to:

- Write a base class and two derived classes that inherit from it using : BaseClass
- Explain the difference between virtual/override and a non-virtual method
- Call the base class constructor from a derived constructor using : base(args)
- Explain what sealed does and when to use it
- Apply the is-a test to decide whether inheritance is appropriate

2.7. Polymorphism (virtual, override)

Builds on: 2.6 (inheritance — polymorphism requires an inheritance relationship), 2.4 (access modifiers — protected and public control what can be overridden), 2.8 (abstract classes — previewed here, covered next).

What is polymorphism and why does it exist?

Polymorphism means 'many forms.' In C#, it means the same code can work with objects of different types, and each type responds in its own way.

```
// Without polymorphism -- fragile, violates open/closed principle:
public void ProcessPayment(object payment)
{
    if (payment is CreditCardPayment cc)
        ChargeCard(cc.CardNumber, cc.Amount);
    else if (payment is BankTransferPayment bt)
        InitiateTransfer(bt.AccountNumber, bt.Amount);
    // Every new payment type requires modifying this method
}

// With polymorphism -- stable, extensible:
public void ProcessPayment(Payment payment)
{
    payment.Execute();    // each type knows how to execute itself
```

```
}
```

How polymorphism works — runtime dispatch

```
public class Payment
{
    public decimal Amount { get; }
    public Payment(decimal amount) => Amount = amount;
    public virtual void Execute()
        => Console.WriteLine($"Processing payment of {Amount:C}");
    public virtual string GetReceipt() => $"Payment: {Amount:C}";
}

public class CreditCardPayment : Payment
{
    public string LastFour { get; }
    public CreditCardPayment(decimal amount, string lastFour) : base(amount)
        => LastFour = lastFour;
    public override void Execute()
        => Console.WriteLine($"Charging card ending {LastFour} for {Amount:C}");
    public override string GetReceipt() => $"Credit card ({LastFour}): {Amount:C}";
}

// All declared as Payment -- the base type:
Payment[] payments = {
    new CreditCardPayment(100m, "4242"),
    new BankTransferPayment(250m, "ACC-001"),
    new Payment(500m)
};

foreach (Payment p in payments)
    p.Execute();

// Output:
// Charging card ending 4242 for $100.00
// Transferring $250.00 to account ACC-001
// Processing payment of $500.00
```

Extending base behaviour with base.Method()

```
public class InternationalOrder : Order
{
    public string DestinationCountry { get; }

    public override void Validate()
    {
        base.Validate(); // run base validation first

        if (string.IsNullOrWhiteSpace(DestinationCountry))
            throw new ArgumentException("Destination country required");
        if (Amount < 100)
            throw new ArgumentException("Minimum international order is $100");
    }
}
```

Polymorphism with collections and services

```
public abstract class DiscountStrategy
{
    public abstract decimal Calculate(decimal orderAmount);
}

public class PercentageDiscount : DiscountStrategy
{
    private readonly decimal _percent;
    public PercentageDiscount(decimal percent) => _percent = percent;
    public override decimal Calculate(decimal orderAmount)
        => orderAmount * (_percent / 100);
}

public class FixedDiscount : DiscountStrategy
{
    private readonly decimal _amount;
    public FixedDiscount(decimal amount) => _amount = amount;
    public override decimal Calculate(decimal orderAmount)
        => Math.Min(_amount, orderAmount);
}

// Pricing service works with ANY discount strategy:
public class PricingService
{
    public decimal ApplyDiscount(decimal amount, DiscountStrategy discount)
        => amount - discount.Calculate(amount);
}
```

Checking type at runtime

```
// is -- check and cast in one:
if (payment is CreditCardPayment cc)
    Console.WriteLine(cc.LastFour);

// switch expression with type patterns -- cleanest form:
string receipt = payment switch
{
    CreditCardPayment cc => $"Card ending {cc.LastFour}",
    BankTransferPayment bt => $"Account {bt.AccountNumber}",
    _ => "Other payment"
};
```

Key Takeaways

- Polymorphism = same code, different behaviour depending on the actual runtime type
- Requires an inheritance relationship (virtual + override or abstract + override)
- The runtime looks at the actual object type, not the declared variable type
- Enables code that works with current and future types without modification
- Reduces type-checking chains — those belong in the class itself
- `base.Method()` extends rather than replaces base behaviour

Keyword	Location	Meaning
---------	----------	---------

virtual	Base class method	Can be overridden by derived classes
override	Derived class	Replaces (or extends) the virtual method
sealed override	Derived class	Overrides and prevents further overriding
base.Method()	Inside override	Calls the base class version
abstract	Base class method	Must be overridden -- no default impl

Memory aid: Polymorphism is like a remote control. The same Play button works on a DVD player, streaming box, and Blu-ray player — each responds in its own way. The remote doesn't know or care which device it's talking to.

Debugging Tips

Symptom	Cause	Fix
Base class method runs	Method not marked virtual	Add virtual to base method
Override not found at runtime	Object was actually base type	Check where object created
new keyword hides instead	Used new instead of override	Replace new with override
Type check chain getting long	Missing virtual method on base	Move to virtual method
base.Method() stack overflow	Circular delegation	Check for circular calls

Self-check — you should now be able to:

- Explain what runtime polymorphism is and how virtual/override enables it
- Write a base class with virtual methods and two derived classes that override them
- Demonstrate that the runtime uses the actual object type, not the variable's declared type
- Explain when base.Method() is appropriate vs completely replacing the implementation
- Apply polymorphism to write code that handles current and future subtypes without modification

2.8. Abstract Classes

Builds on: 2.6 (inheritance — abstract classes are inherited), 2.7 (polymorphism — abstract methods are the strongest form of virtual), 2.4 (access modifiers — protected members are common in abstract classes).

What are abstract classes and why do they exist?

In 2.7 you saw virtual methods — a base class provides a default implementation that derived classes can override. But sometimes there is no sensible default. The base class knows a method must exist, but has no idea how to implement it.

Abstract classes solve this: they define a contract ('you must implement these methods') while still providing shared state and behaviour that all derived classes get for free.

```
// virtual -- base provides default, override is optional:
public virtual decimal Calculate() => 0m;
```

```
// abstract -- base provides NO implementation, override is mandatory:
public abstract decimal Calculate();
```

An abstract class: cannot be instantiated directly, can have abstract methods (declared but not implemented), can have concrete methods (fully implemented and shared), can have fields, properties, and constructors.

Defining an abstract class

```
public abstract class TaxCalculator
{
    public string Region { get; }
    public decimal TaxRate { get; protected set; }

    protected TaxCalculator(string region, decimal taxRate)
    {
        Region = region;
        TaxRate = taxRate;
    }

    // Abstract method -- MUST be implemented:
    public abstract decimal Calculate(decimal amount);
    public abstract string TaxName { get; }

    // Concrete method -- shared by all derived classes:
    public string GetSummary(decimal amount)
    {
        var tax = Calculate(amount);
        return $"{TaxName} ({Region}): {tax:C} on {amount:C}";
    }

    public virtual bool AppliesTo(string productCategory) => true;
}

var calc = new TaxCalculator("India", 0.18m);    // compile error -- cannot instantiate

public class GstCalculator : TaxCalculator
{
    public GstCalculator() : base("India", 0.18m) { }
    public override decimal Calculate(decimal amount) => amount * TaxRate;
    public override string TaxName => "GST";
}

var gst = new GstCalculator();
Console.WriteLine(gst.GetSummary(1000m));
// "GST (India): $180.00 on $1,000.00"
```

Abstract class vs interface — the key distinction

Use abstract class when: derived classes share state (fields), share concrete behaviour, there's a genuine is-a relationship, you want to control construction, or you need protected members.

Use interface when: defining a pure capability contract with no shared state, multiple unrelated types need to satisfy the contract, or you need multiple contracts on one type.

```
// Abstract class -- shared state + behaviour + contract:
public abstract class BaseRepository<T>
{
```

```

protected readonly DbContext _context;

protected BaseRepository(DbContext context) { _context = context; }

// Shared concrete behaviour:
public async Task<T?> GetByIdAsync(int id)
    => await _context.Set<T>().FindAsync(id);

public async Task SaveAsync() => await _context.SaveChangesAsync();

// Contract -- derived classes define their own queries:
public abstract Task<IEnumerable<T>> GetAllAsync();
}

```

Template Method pattern

```

public abstract class ReportGenerator
{
    // Template method -- algorithm skeleton, not overridable:
    public sealed Report Generate(ReportRequest request)
    {
        var data = FetchData(request);
        var processed = ProcessData(data);
        var formatted = FormatOutput(processed);
        AddHeader(formatted, request);
        AddFooter(formatted);
        return new Report(formatted);
    }

    protected abstract IEnumerable<object> FetchData(ReportRequest request);
    protected abstract IEnumerable<object> ProcessData(IEnumerable<object> data);
    protected abstract string FormatOutput(IEnumerable<object> data);

    private void AddHeader(string content, ReportRequest request)
        => Console.WriteLine($"=== Report: {request.Title} ===");
    private void AddFooter(string content)
        => Console.WriteLine($"Generated: {DateTime.UtcNow}");
}

```

Background service — abstract class in ASP.NET Core

```

// ASP.NET Core provides BackgroundService as an abstract class:
public class EmailQueueProcessor : BackgroundService
{
    // Must override this abstract method:
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await ProcessNextEmailAsync();
            await Task.Delay(1000, stoppingToken);
        }
    }
}

```


Key Takeaways

- An abstract class cannot be instantiated — only its concrete subclasses can
- Abstract methods have no implementation — derived classes must override them
- Concrete methods in abstract classes are shared by all derived classes
- Abstract classes are the right choice when derived classes share state and partial behaviour
- The Template Method pattern — define algorithm skeleton in base, fill steps in derived
- Use interface when you only need a contract with no shared state

Feature	abstract method	virtual method	Interface method
Has implementation	No	Yes (default)	No (usually)
Must override	Yes	No	Yes
Class instantiated	No	Yes	N/A
Can have fields	Yes	Yes	No
Multiple per class	No	No	Yes

Memory aid: An abstract class is an unfinished blueprint. The architect drew the floor plan — but left blank spaces where each unit decides its own interior design. You cannot live in the blueprint itself — only in a finished unit.

Debugging Tips

Symptom	Cause	Fix
Cannot create instance of abstract	Trying to new an abstract class	Instantiate concrete derived
Derived class won't compile	Not all abstract members impl	Implement every abstract mem
Abstract method has a body	abstract methods can't have body	Remove body -- end with ;
Template method calls wrong ver	Check which concrete is created	Verify with GetType().Name
EF can't materialise abstract	Querying abstract type directly	Query concrete type

Self-check — you should now be able to:

- Explain the difference between abstract and virtual and when to choose each
- Write an abstract class with a mix of abstract methods, concrete methods, and shared state
- Explain why an abstract class cannot be instantiated and what that forces you to do
- Apply the Template Method pattern
- Decide between abstract class and interface given a design scenario

2.9. Interfaces (including Default Interface Methods)

Builds on: 2.7 (polymorphism — interfaces are the other polymorphism mechanism), 2.8 (abstract classes — interfaces vs abstract classes is a core design decision), 2.4 (access modifiers — interface members are implicitly public).

What are interfaces and why do they exist?

An abstract class defines a partial contract with some shared state and behaviour. An interface defines a pure contract — just a set of capabilities, with no state and (traditionally) no implementation.

The problem interfaces solve is deeper than abstract classes. C# has single inheritance — a class can only have one base class. But a class can implement many interfaces.

```
// A class can only have ONE base class:
public class OrderService : BaseService

// But can implement MANY interfaces:
public class OrderService : IOrderService, IDisposable, IValidatable, ILoggable
```

Defining an interface

```
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(int id);
    Task<IEnumerable<Order>> GetAllAsync();
    Task<Order> CreateAsync(Order order);
    Task UpdateAsync(Order order);
    Task DeleteAsync(int id);
}
```

Implementing an interface

```
public class SqlOrderRepository : IOrderRepository
{
    private readonly AppDbContext _context;

    public SqlOrderRepository(AppDbContext context) { _context = context; }

    public async Task<Order?> GetByIdAsync(int id)
        => await _context.Orders.FindAsync(id);

    public async Task<IEnumerable<Order>> GetAllAsync()
        => await _context.Orders.ToListAsync();

    public async Task<Order> CreateAsync(Order order)
    {
        _context.Orders.Add(order);
        await _context.SaveChangesAsync();
        return order;
    }

    public async Task UpdateAsync(Order order)
    {
        _context.Orders.Update(order);
        await _context.SaveChangesAsync();
    }

    public async Task DeleteAsync(int id)
    {
        var order = await GetByIdAsync(id);
        if (order != null)
        {
            _context.Orders.Remove(order);
            await _context.SaveChangesAsync();
        }
    }
}
```

```
    }
}
```

Interface inheritance

```
public interface IReadRepository<T>
{
    Task<T?> GetByIdAsync(int id);
    Task<IEnumerable<T>> GetAllAsync();
}

public interface IWriteRepository<T>
{
    Task<T> CreateAsync(T entity);
    Task UpdateAsync(T entity);
    Task DeleteAsync(int id);
}

// Combines both:
public interface IRepository<T> : IReadRepository<T>, IWriteRepository<T> { }
```

Explicit interface implementation

```
public interface IPrintable { void Print(); }
public interface IExportable { void Print(); }    // same method name

public class Document : IPrintable, IExportable
{
    void IPrintable.Print() => Console.WriteLine("Printing document...");
    void IExportable.Print() => Console.WriteLine("Exporting document...");
}

var doc = new Document();
doc.Print();    // compile error -- not on public surface

((IPrintable)doc).Print();    // "Printing document..."
((IExportable)doc).Print();    // "Exporting document..."
```

Default interface methods — C# 8+

```
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(int id);
    Task<IEnumerable<Order>> GetAllAsync();

    // Default implementation -- existing classes don't need to implement:
    async Task<bool> ExistsAsync(int id)
        => await GetByIdAsync(id) != null;
}

// Classes that don't override get the default automatically
// But: default methods are only accessible through the interface reference!

var repo = new SqlOrderRepository(context);
repo.ExistsAsync(1);    // compile error if SqlOrderRepository didn't override
```

```
IOrderRepository irepo = repo;
irepo.ExistsAsync(1); // accessible through interface reference
```

Interfaces in dependency injection — the core pattern

```
// Define contracts:
public interface IOrderService { Task<Order?> GetByIdAsync(int id); }
public interface IEmailService { Task SendAsync(string to, string subject); }

// Consume contracts -- never concrete types:
public class CheckoutService
{
    private readonly IOrderService _orders;
    private readonly IEmailService _email;

    public CheckoutService(IOrderService orders, IEmailService email)
    {
        _orders = orders;
        _email = email;
    }
}

// Register in Program.cs:
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<IEmailService, SmtEmailService>();

// In tests -- swap implementations:
var mockEmail = new Mock<IEmailService>();
var service = new CheckoutService(mockOrders, mockEmail.Object);
```

Key Takeaways

- An interface is a pure contract — no state, no constructor, just signatures
- A class can implement many interfaces but inherit only one class
- Interface references enable polymorphism — any implementing type satisfies the contract
- Dependency injection in .NET is built on interfaces — register interface -> implementation, consume interface
- Default interface methods (C# 8+) allow interface evolution — but only accessible through interface reference
- Explicit implementation resolves name conflicts and hides members from the concrete type

	Interface	Abstract Class
-----	+-----	+-----
Fields	No	Yes
Constructor	No	Yes (protected)
Default impl	Yes (C# 8+)	Yes (full)
Multiple per class	Yes	No (one only)
Access modifiers	No (pub implicit)	Yes
Best for	Contracts	Shared state+template

.NET-specific rules:

- Interface names start with I by convention — always (IOrderService, IRepository)

- Register interfaces in DI with AddScoped, AddTransient, or AddSingleton
- IDisposable — implement whenever your class holds unmanaged resources
- Default interface methods should be used for interface evolution only — not as a design-first choice

Memory aid: An interface is a job description. It says what you must be able to do. A class is the employee who fulfils that description — and one employee can hold multiple job titles.

Debugging Tips

Symptom	Cause	Fix
does not implement interface mbr	Missing method implementations	Implement all members
Default method not on concrete	Only via interface reference	Cast to interface or override
DI throws Unable to resolve IXxx	Not registered in Program.cs	Add AddScoped<IXxx, ConcreteX
Explicit impl not accessible	Hidden from concrete type	Access through interface var
Mock doesn't intercept calls	Creating concrete directly	Inject through interface

Self-check — you should now be able to:

- Write an interface and two implementations that can be swapped via DI
- Explain why interfaces are central to testability and dependency injection in .NET
- Implement multiple interfaces on a single class
- Use explicit interface implementation to resolve name conflicts
- Explain what default interface methods are, their key limitation, and when to use them

2.10. Sealed Classes and Methods

Builds on: 2.6 (inheritance — sealed stops it), 2.7 (polymorphism — sealed stops override chains), 2.8 (abstract classes — the opposite end of the spectrum from sealed).

What does sealed do and why does it exist?

sealed prevents inheritance or further overriding. It's the explicit statement: 'this class is complete — it is not designed to be a base class' or 'this override is final — no further specialisation is allowed.'

Two motivations: Design intent — some classes represent a complete, specific concept with no meaningful variations. Performance — the JIT compiler can optimise calls to sealed methods more aggressively because it knows the exact implementation at compile time.

Sealed classes

```
public sealed class SqlConnectionString
{
    public string Server { get; }
    public string Database { get; }
    public int Port { get; }
    public bool Encrypt { get; }

    public SqlConnectionString(string server, string database,
```

```

        int port = 1433, bool encrypt = true)
    {
        Server = server;
        Database = database;
        Port = port;
        Encrypt = encrypt;
    }

    public override string ToString()
        => $"Server={Server},{Port};Database={Database};Encrypt={Encrypt}";
}

public class CustomConnectionString : SqlConnectionString { }    // compile error

```

Sealed methods — sealing a specific override

```

public class CreditCardPayment : Payment
{
    public override void Validate() { base.Validate(); }

    // Seal this override:
    public sealed override string GetReceipt()
        => $"Credit card payment: {Amount:C} on card ending {LastFour}";
}

public class PremiumCreditCardPayment : CreditCardPayment
{
    public override void Validate() { ... }    // fine -- not sealed
    public override string GetReceipt() { ... }    // compile error -- sealed
}

```

Sealed in domain-driven design

```

public sealed class Money
{
    public decimal Amount { get; }
    public string Currency { get; }

    public Money(decimal amount, string currency)
    {
        if (amount < 0) throw new ArgumentException("Amount cannot be negative");
        Amount = amount;
        Currency = currency.ToUpperInvariant();
    }

    public Money Add(Money other)
    {
        if (Currency != other.Currency) throw new InvalidOperationException();
        return new Money(Amount + other.Amount, Currency);
    }

    public override string ToString() => $"{Amount:F2} {Currency}";
}

public sealed class EmailAddress

```

```

{
    public string Value { get; }
    public EmailAddress(string value)
    {
        if (string.IsNullOrEmpty(value) || !value.Contains('@'))
            throw new ArgumentException("Invalid email address");
        Value = value.ToLowerInvariant().Trim();
    }
}

```

Sealed + private constructor — Singleton pattern

```

public sealed class Singleton
{
    private Singleton() { }
    public static Singleton Instance { get; } = new Singleton();
}

```

Sealed record hierarchies

```

public abstract record Shape;
public sealed record Circle(double Radius) : Shape;
public sealed record Rectangle(double Width, double Height) : Shape;

// Compiler can verify all cases are handled:
double area = shape switch
{
    Circle c    => Math.PI * c.Radius * c.Radius,
    Rectangle r => r.Width * r.Height,
    // No _ needed -- compiler knows these are all possible types
};

```

Key Takeaways

- sealed on a class prevents any inheritance — it is a final, complete type
- sealed override on a method prevents further overriding of that specific method
- Structs are implicitly sealed — you cannot inherit from a struct
- sealed communicates design intent: 'this type is not designed to be extended'
- Value objects, identifiers, and leaf exception types are natural candidates for sealed

Usage	Syntax	Effect
Seal a class	<code>public sealed class Foo</code>	No class can inherit Foo
Seal a method ovrd	<code>public sealed override void Bar</code>	No further override of Bar
Struct (implicit)	<code>public struct Foo</code>	Always sealed
Record struct	<code>public record struct Foo</code>	Always sealed

Memory aid: sealed is a dead-end sign on a road. You can drive up to it, but you cannot continue past it — and no other road can branch from it.

Debugging Tips

Symptom	Cause	Fix
-----	-----	-----

cannot derive from sealed type	Inheriting a sealed class	Remove inheritance, use composition
cannot override sealed member	Sealed override is final	Remove the override
Struct inheritance attempt fails	Structs are implicitly sealed	Use class or interface instead

Self-check — you should now be able to:

- Apply sealed to a class and explain what it prevents
- Apply sealed override to a method and explain the difference from sealing the whole class
- Explain why structs are always sealed and what that means
- Identify types in your own domain that are appropriate candidates for sealed
- Use sealed record hierarchies with switch expressions for exhaustive pattern matching

2.11. Static Classes and Members

Builds on: 2.1 (classes — static is a modifier on class members), 2.2 (constructors — static constructors are a special form), 1.18 (const/readonly — static readonly is the runtime constant pattern), 1.19 (scope — static fields live for the entire app lifetime).

What are static classes and members and why do they exist?

Most code you write is instance-based. But some things genuinely don't belong to any specific object: mathematical operations, pure utility functions, factory methods, extension methods.

Static members and static classes exist for these stateless, instance-independent scenarios.

Static members — the basics

```
public class OrderCounter
{
    private static int _totalOrders = 0;    // one copy for all instances
    public int Id { get; }

    public OrderCounter()
    {
        _totalOrders++;
        Id = _totalOrders;
    }

    public static int GetTotalOrders() => _totalOrders;
    public string GetInfo() => $"Order #{Id} of {_totalOrders}";
}

var a = new OrderCounter();    // _totalOrders = 1
var b = new OrderCounter();    // _totalOrders = 2
Console.WriteLine(OrderCounter.GetTotalOrders());    // 3
Console.WriteLine(a.GetInfo());    // "Order #1 of 3"
```

Static classes — purely utility, no instances

```
public static class MathHelpers
{
```



```

    public static decimal RoundToNearest(decimal value, decimal nearest)
        => Math.Round(value / nearest) * nearest;

    public static decimal Clamp(decimal value, decimal min, decimal max)
        => Math.Max(min, Math.Min(max, value));

    public static bool IsBetween(decimal value, decimal min, decimal max)
        => value >= min && value <= max;
}

decimal price = MathHelpers.RoundToNearest(9.87m, 0.25m);    // 10.00
decimal clamped = MathHelpers.Clamp(150m, 0m, 100m);        // 100

```

Extension methods — the most important use of static classes

```

public static class StringExtensions
{
    public static bool IsValidEmail(this string value)
        => !string.IsNullOrEmpty(value) && value.Contains('@') && value.Contains('.');

    public static string Truncate(this string value, int maxLength, string suffix = "...")
    {
        if (string.IsNullOrEmpty(value) || value.Length <= maxLength) return value;
        return value[..maxLength] + suffix;
    }

    public static string ToTitleCase(this string value)
    {
        if (string.IsNullOrEmpty(value)) return value;
        return System.Globalization.CultureInfo.CurrentCulture
            .TextInfo.ToTitleCase(value.ToLower());
    }
}

"s@zoho.com".IsValidEmail()    // true
"Hello World".Truncate(5)      // "Hello..."
"hello world".ToTitleCase()    // "Hello World"

```

IQueryable extension methods for EF Core

```

public static class OrderQueryExtensions
{
    public static IQueryable<Order> Active(this IQueryable<Order> query)
        => query.Where(o => o.Status != "Cancelled" && o.Status != "Deleted");

    public static IQueryable<Order> ForUser(this IQueryable<Order> query, int userId)
        => query.Where(o => o.UserId == userId);

    public static IQueryable<Order> Recent(this IQueryable<Order> query, int days = 30)
        => query.Where(o => o.CreatedAt >= DateTime.UtcNow.AddDays(-days));

    public static IQueryable<Order> Paginate(
        this IQueryable<Order> query, int page, int pageSize)
        => query.Skip((page - 1) * pageSize).Take(pageSize);
}

```

```
// Usage in controller:
var orders = await _context.Orders
    .Active()
    .ForUser(userId)
    .Recent()
    .Paginate(page, pageSize)
    .ToListAsync();
```

Warning — static mutable state in ASP.NET Core

```
public class OrderService
{
    // DANGEROUS in ASP.NET Core -- shared across all concurrent requests:
    private static List<Order> _pendingOrders = new();

    // SAFE -- instance field, each request gets its own OrderService:
    private List<Order> _pendingOrders = new();
}
```

Key Takeaways

- Static members belong to the type, not any instance — one shared copy for the entire application
- Static classes are sealed, cannot be instantiated, and can only contain static members
- Extension methods must be in static classes — the primary reason you write them
- Static fields in ASP.NET Core are shared across all concurrent requests — use carefully
- Factory methods, helpers, and utilities are appropriate as static members
- Prefer DI-managed singletons over static mutable fields for application-wide state

	Static class	Static member in regular class
Instantiation	Never	Class can be instantiated
Inheritance	Cannot	Class can be inherited
Instance members	None allowed	Both instance and static
Primary use	Utilities, extns	Shared counters, factories

Memory aid: Static = carved in stone on the building itself. Instance = written on sticky notes attached to each tenant. The building (type) has one stone inscription. Each tenant (object) has their own sticky notes.

Debugging Tips

Symptom	Cause	Fix
Extension method not found	Namespace not imported	Add using for extension class
Static field causes data bleed	Mutable static shared across reqs	Move to instance or DI Scope
Race condition on static counter	Unsynchronised ++ from threads	Use Interlocked.Increment
cannot declare instance members	In a static class	Move to regular class
HttpClient exhausting sockets	new HttpClient() per request	Use static readonly or IHT

Self-check — you should now be able to:

- Explain the difference between a static member and an instance member

- Write a static class with extension methods and use them via dot notation
- Write a static factory method on a non-static class
- Explain why mutable static state is dangerous in ASP.NET Core
- Identify where extension methods are used in everyday .NET code (LINQ, claim helpers, query filters)

2.12. Partial Classes

Builds on: 2.1 (classes — partial is a modifier on a class definition), 1.2 (project structure — partial classes span multiple files in the same project), 2.11 (static — code generators use partial classes extensively).

What are partial classes and why do they exist?

A partial class splits a single class definition across multiple files. At compile time, all the parts are merged into one class — as if they were written together.

Two problems this solves: Code generation — tools like EF Core migrations, scaffolding, Blazor, and WinForms generate code automatically. Partial classes let the generator own one file and you own another. Large classes in specific scenarios — some genuinely large classes can be split by concern across files without creating artificial sub-types.

Basic syntax

```
// Order.cs -- your business logic:
public partial class Order
{
    public int Id { get; private set; }
    public string Status { get; private set; } = "Pending";
    public decimal Amount { get; private set; }

    public Order(int id, decimal amount) { Id = id; Amount = amount; }

    public void Cancel()
    {
        if (Status == "Shipped")
            throw new InvalidOperationException("Cannot cancel shipped order");
        Status = "Cancelled";
    }
}

// Order.Validation.cs -- validation logic separated by concern:
public partial class Order
{
    public bool IsValid() => Amount > 0 && !string.IsNullOrEmpty(Status);

    public IEnumerable<string> GetValidationErrors()
    {
        if (Amount <= 0) yield return "Amount must be positive";
        if (string.IsNullOrEmpty(Status)) yield return "Status is required";
    }
}
```

```

}

// Order.Mapping.cs -- mapping logic separated:
public partial class Order
{
    public OrderDto ToDto()
        => new OrderDto { Id = Id, Status = Status, Amount = Amount };
}

```

Rules for partial classes

```

// All parts must agree:
public partial class Order { }      // file 1
public partial class Order { }      // file 2 -- same access modifier

// Attributes accumulate across parts:
[SomeAttribute] public partial class Order { }    // file 1
[AnotherAttribute] public partial class Order { } // file 2
// Result: Order has both attributes

// Interfaces accumulate:
public partial class Order : IValidatable { }     // file 1
public partial class Order : IDisposable { }      // file 2
// Result: Order implements both

```

The primary real-world use — code generation

```

// AppDbContext.cs -- YOUR file:
public partial class AppDbContext : DbContext
{
    public DbSet<Order> Orders { get; set; }
    public DbSet<Product> Products { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
        => modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
}

// AppDbContext.Designer.cs -- GENERATED file, never edit:
public partial class AppDbContext
{
    protected override void OnConfiguring(DbContextOptionsBuilder options)
    {
        if (!options.IsConfigured)
            options.UseSqlServer("your-connection-string");
    }
}

```

Partial methods — the handshake pattern

```

// Generated part -- declares the hook:
public partial class Order
{
    partial void OnStatusChanged(string oldStatus, string newStatus);

    public void SetStatus(string newStatus)
    {
        OnStatusChanged(OldStatus, newStatus);
    }
}

```

```

    {
        var old = Status;
        Status = newStatus;
        OnStatusChanged(old, newStatus);    // silently removed if not implemented
    }
}

// Your part -- optionally implement the hook:
public partial class Order
{
    partial void OnStatusChanged(string oldStatus, string newStatus)
    {
        Console.WriteLine($"Order {Id} status: {oldStatus} -> {newStatus}");
    }
}

```

If you don't implement `OnStatusChanged`, the call in `SetStatus` is compiled away entirely — zero overhead. C# 9+ partial methods with return types or access modifiers require implementation.

When to use partial classes

Use: Code generation / source generators -- primary purpose
 Separation of generated code from hand-written code
 Razor/Blazor code-behind (.razor.cs pattern)
 Very large classes with genuinely distinct concerns

Avoid: Splitting a class just because it got large -- fix the design
 Using partial as a workaround for missing abstraction
 Sharing behaviour between unrelated classes

Realistic scenario — DbContext with partial separation

```

// AppDbContext.cs -- DbSet and model configuration:
public partial class AppDbContext : DbContext
{
    public DbSet<Order> Orders { get; set; }
    public DbSet<Product> Products { get; set; }
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
    protected override void OnModelCreating(ModelBuilder builder)
        => builder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
}

// AppDbContext.SeedData.cs -- seeding logic:
public partial class AppDbContext
{
    public async Task SeedAsync()
    {
        if (!await Products.AnyAsync())
        {
            Products.AddRange(GetSeedProducts());
            await SaveChangesAsync();
        }
    }

    private IEnumerable<Product> GetSeedProducts() =>

```

```
[
    new Product { Name = "Widget", Price = 9.99m },
    new Product { Name = "Gadget", Price = 24.99m },
];
}
```

Key Takeaways

- Partial classes split one class definition across multiple files — compiled into one class
- The primary purpose is code generation — separating generated code from hand-written code
- All parts must be in the same namespace and assembly, and must agree on access modifiers
- Partial methods are optional hooks — declared in one part, optionally implemented in another
- C# 9+ partial methods with return types or access modifiers require implementation
- Never edit generated partial class files — they'll be overwritten

Rule	Detail
Same namespace	All parts must be in the same namespace
Same assembly	Cannot span projects
Access modifier	Must match across all parts
Base class	One part specifies it
Interfaces	Accumulate -- each part can add more
Attributes	Accumulate -- each part can add more

Memory aid: Partial classes are like a jigsaw puzzle. Each file is one piece. The compiler assembles them into the complete picture at build time. You paint your piece; the generator paints its piece.

Debugging Tips

Symptom	Cause	Fix
partial modifier syntax error	partial must precede class	public partial class, not p
Member from one part not found	Namespace mismatch	Ensure both files same name
Generated file overwrites changes	Edited generated file	Move code to hand-written p
Partial method impl not compiling	Return type on pre-C# 9 method	Make void and private
Two parts disagree on base class	Different base in each	One file declares base, oth

Self-check — you should now be able to:

- Explain what a partial class is and the compiler-level mechanism behind it
- Identify why partial classes exist and what their primary purpose is in .NET
- Write a partial class split across two files and explain what the compiler does
- Explain partial methods — declaration, optional implementation, and what happens when not implemented
- Explain why you should never edit generated partial class files

2.13. Records (record class, record struct, with expressions)

Builds on: 2.1 (classes — records are a class variant), 2.3 (properties — records auto-generate init-only properties), 1.16 (structs — record struct combines struct semantics with record features), 1.22 (tuples — records solve similar problems but with named types).

What are records and why do they exist?

Before records, modelling immutable data in C# required significant boilerplate — constructors, Equals, GetHashCode, ToString, all written manually. Records were introduced in C# 9 specifically for immutable data modelling — DTOs, value objects, API responses, domain events, query results.

```
// One line replaces all the boilerplate:
public record OrderSummary(int Id, string CustomerName, decimal Amount);
```

This one line generates: a constructor with all three parameters, public init-only properties, Equals (structural/value-based), GetHashCode, ToString, Deconstruct, and a copy constructor for with expressions.

Structural equality — the key difference from classes

```
var a = new OrderSummary(1, "Sathish", 999.99m);
var b = new OrderSummary(1, "Sathish", 999.99m);
var c = new OrderSummary(2, "Priya", 500.00m);

Console.WriteLine(a == b);    // true -- same values
Console.WriteLine(a == c);    // false -- different values
Console.WriteLine(a);         // OrderSummary { Id = 1, CustomerName = Sathish, Amount = 999.99 }

var (id, name, amount) = a;    // deconstruction works automatically

// Compare with a regular class:
var x = new RegularClass(1, "Sathish");
var y = new RegularClass(1, "Sathish");
Console.WriteLine(x == y);    // false -- different references
```

with expressions — non-destructive mutation

```
public record Order(int Id, string Status, decimal Amount, string CustomerName);

var original = new Order(1, "Pending", 999.99m, "Sathish");

// Create a new Order with only Status changed:
var shipped = original with { Status = "Shipped" };

// Create another with multiple changes:
var amended = original with { Status = "Processing", Amount = 1099.99m };

Console.WriteLine(original.Status);    // "Pending" -- unchanged
Console.WriteLine(shipped.Status);    // "Shipped"
Console.WriteLine(amended.Status);    // "Processing"
Console.WriteLine(amended.CustomerName); // "Sathish" -- copied from original
```

Adding members to records

```
public record Money(decimal Amount, string Currency)
{
    // Computed property:
    public bool IsZero => Amount == 0;

    // Method:
    public Money Add(Money other)
    {
        if (Currency != other.Currency)
            throw new InvalidOperationException("Currency mismatch");
        return this with { Amount = Amount + other.Amount };
    }

    // Compact constructor -- validation without repeating parameters:
    public Money
    {
        if (Amount < 0) throw new ArgumentException("Amount cannot be negative");
        if (string.IsNullOrEmpty(Currency)) throw new ArgumentException("Currency required");
        Currency = Currency.ToUpperInvariant();
    }

    public override string ToString() => $"{Amount:F2} {Currency}";
}

var price = new Money(100m, "usd");    // Currency normalised to "USD"
var tax   = new Money(18m, "USD");
var total = price.Add(tax);
Console.WriteLine(total);    // "118.00 USD"
```

record struct — value type record

```
public record struct Point(int X, int Y);
public record struct DateRange(DateTime Start, DateTime End);

var p1 = new Point(1, 2);
var p2 = new Point(1, 2);
var p3 = p1;    // copy -- independent

p3 = p3 with { X = 99 };

Console.WriteLine(p1 == p2);    // true -- value equality
Console.WriteLine(p1.X);        // 1 -- unchanged
Console.WriteLine(p3.X);        // 99
Console.WriteLine(p1);           // "Point { X = 1, Y = 2 }"

// readonly record struct -- fully immutable:
public readonly record struct Money(decimal Amount, string Currency);
```

Record inheritance

```
public abstract record Shape(string Colour);
public record Circle(string Colour, double Radius) : Shape(Colour);
public record Rectangle(string Colour, double Width, double Height) : Shape(Colour);
```



```

Shape shape = new Circle("red", 5.0);
Console.WriteLine(shape);    // "Circle { Colour = red, Radius = 5 }"

// Sealed records with switch:
double area = shape switch
{
    Circle c    => Math.PI * c.Radius * c.Radius,
    Rectangle r => r.Width * r.Height,
};

```

Records in real .NET API code

```

// API response DTOs:
public record OrderResponse(
    int Id, string CustomerName, decimal Amount,
    string Status, DateTime CreatedAt);

// Domain events:
public abstract record DomainEvent(DateTime OccurredAt);
public sealed record OrderPlaced(int OrderId, string CustomerEmail,
    decimal Amount, DateTime OccurredAt) : DomainEvent(OccurredAt);
public sealed record OrderShipped(int OrderId, string TrackingNumber,
    DateTime OccurredAt) : DomainEvent(OccurredAt);

// Pagination:
public record PagedResult<T>(IReadOnlyList<T> Items, int TotalCount, int Page, int PageSize)
{
    public int TotalPages => (int)Math.Ceiling(TotalCount / (double)PageSize);
    public bool HasNext => Page < TotalPages;
    public bool HasPrevious => Page > 1;
}

```

Key Takeaways

- Records are for immutable data modelling — value-based equality, clean syntax
- record class (just record) — reference type, structural equality, immutable by default
- record struct — value type, same features as record class but stack-allocated
- readonly record struct — fully immutable value type
- with expressions create a new record with specified properties changed — original unchanged
- Compact constructors allow validation/transformation without repeating parameter lists
- EF Core entities should be classes, not records — EF needs mutable state

	record/record class	record struct	readonly record struct
Type	Reference	Value	Value
Equality	Structural	Structural	Structural
Mutable by default	No	Yes	No
Inheritance	Yes (from records)	No	No
with support	Yes	Yes	Yes
Stack allocated	No	Yes	Yes

Memory aid: A record is a snapshot — a photo of data at a point in time. You don't edit the photo. If things change, you take a new photo (with). Two photos of the same scene are equal, regardless of which camera took them.

Debugging Tips

Symptom	Cause	Fix
Record properties can't be set	init-only -- intentional	Use with, or get; set; if
Two records not equal	One is derived type of the other	Ensure both are exact sam
EF Core fails to track record	Records are immutable	Use a class for EF entiti
with expression compile error	Only works on records/structs	Convert to record
JSON deserialization null props	Parameter names don't match JSON	Use [JsonPropertyName] a

Self-check — you should now be able to:

- Create a positional record and explain every member it auto-generates
- Use a with expression to create a modified copy and explain why the original is unchanged
- Explain the difference between record, record struct, and readonly record struct
- Explain why records compare by value while regular classes compare by reference
- Write a compact constructor for validation in a record

2.14. Object Initializers

Builds on: 2.1 (classes — object initializers set class properties), 2.2 (constructors — object initializers run after the constructor), 2.3 (properties — only settable properties work with object initializers), 2.13 (records — init-only properties are designed for object initializers).

What are object initializers and why do they exist?

Without object initializers, setting multiple properties after construction required separate assignment statements. Object initializers collapse this into a single expression — cleaner, composable, and usable in contexts where separate statements aren't allowed (LINQ projections, field initializers, return statements).

```
// Old way:
var order = new Order();
order.Id = 1;
order.CustomerName = "Sathish";
order.Amount = 999.99m;

// With object initializer:
var order = new Order
{
    Id = 1,
    CustomerName = "Sathish",
    Amount = 999.99m,
    Status = "Pending",
    CreatedAt = DateTime.UtcNow
};
```

How object initializers work

```
// What you write:
var order = new Order { Id = 1, Amount = 500m };

// What the compiler generates:
var __temp = new Order();
__temp.Id = 1;
__temp.Amount = 500m;
var order = __temp;
```

The constructor runs first. The object initializer sets properties after the constructor has already completed. The reference isn't assigned until the initializer completes.

What properties can be set

```
public class User
{
    public int Id { get; set; }           // public set -- works
    public string Email { get; init; }    // init-only -- works in initializer
    public string Role { get; private set; } // private set -- NOT accessible
    public string Hash { get; }          // no setter -- NOT accessible
    public int Age => 30;                 // computed -- NOT accessible
}

var user = new User
{
    Id = 1,                             // works
    Email = "s@z.com",                  // works -- init-only allowed here
    Role = "Admin",                     // compile error
};
```

Nested object initializers

```
var order = new Order
{
    Id = 1,
    ShippingAddress = new Address
    {
        Street = "42 MG Road",
        City = "Chennai",
        PostCode = "600001"
    },
    Customer = new CustomerInfo
    {
        Name = "Sathish",
        Email = "s@zoho.com"
    }
};
```

Collection initializers

```
// List initializer:
var tags = new List<string> { "dotnet", "csharp", "api" };
```

```
// Dictionary index initializer:
var codes = new Dictionary<int, string>
{
    [200] = "OK",
    [404] = "Not Found",
    [500] = "Internal Server Error"
};

// Nested -- orders with line items:
var order = new Order
{
    Id = 1,
    Items = new List<OrderItem>
    {
        new OrderItem { ProductId = 1, Quantity = 2, Price = 99.99m },
        new OrderItem { ProductId = 2, Quantity = 1, Price = 149.99m }
    }
};
```

required properties — C# 11

```
public class CreateOrderDto
{
    public required string CustomerName { get; init; }
    public required decimal Amount { get; init; }
    public string? Notes { get; init; } // optional
    public string Currency { get; init; } = "USD"; // has default
}

var dto = new CreateOrderDto(); // compile error -- required not set

var dto = new CreateOrderDto
{
    CustomerName = "Sathish",
    Amount = 500m
    // Notes omitted -- fine, not required
};
```

Object initializers in LINQ

```
var orderDtos = orders
    .Where(o => o.IsActive)
    .Select(o => new OrderDto
    {
        Id = o.Id,
        CustomerName = o.CustomerName,
        Amount = o.Amount,
        Status = o.Status,
        ItemCount = o.Items.Count,
        CreatedAt = o.CreatedAt
    })
    .OrderByDescending(dto => dto.CreatedAt)
    .ToList();
```

Target-typed new — C# 9

```
// Full syntax:
Order order = new Order { Id = 1, Amount = 500m };

// Target-typed -- type inferred from the variable:
Order order = new() { Id = 1, Amount = 500m };

// In collection initializers:
var orders = new List<Order>
{
    new() { Id = 1, Amount = 100m },
    new() { Id = 2, Amount = 200m },
};
```

Key Takeaways

- Object initializers set properties after the constructor runs — in a single expression
- Only properties with accessible setters (set or init) can be initialized this way
- init-only properties can be set in an object initializer but not after construction
- Collection initializers compile to Add calls
- required properties (C# 11) must be set in the initializer — compile error if omitted
- Target-typed new() omits the type name when it's inferrable from context
- Object initializers are essential in LINQ projections and test data setup

Syntax	In initializer	After construction?
-----+-----+-----		
{ get; set; }	Yes	Yes
{ get; init; }	Yes	No
{ get; private set; }	No	No (only in class)
{ get; }	No	No
required init	Yes (mandatory)	No

Memory aid: An object initializer is like filling out a form immediately after it's handed to you. The form (constructor) gives you the blank template; you fill in the fields (initializer) before you hand it back.

Debugging Tips

Symptom	Cause	Fix
-----+-----+-----		
Property set in initializer null	Property has get; only	Add init or set accessor
init-only set outside initializer	Post-construction mutation	Use with (records) or set
required property error	Forgot required in initializer	Set all required properties
Nested object is null	Forgot new Type {}	Add nested initializer
JSON deserialization gives null	Only constructor, no setters	Add { get; set; } or init

Self-check — you should now be able to:

- Explain what an object initializer is and what the compiler generates from it
- Identify which property accessor types work in object initializers and which don't
- Use nested object initializers and collection initializers to build complex object graphs

- Apply required to enforce that certain properties are always provided
- Explain why object initializers are essential in LINQ Select projections

2.15. The this Keyword

Builds on: 2.1 (classes — this refers to the current instance), 2.2 (constructors — this() chains constructors), 2.3 (properties — this disambiguates property vs parameter), 2.11 (extension methods — this marks the extended type).

What is this and why does it exist?

this is a reference to the current instance — the specific object whose method or constructor is currently executing. It exists to:

- Disambiguate when a parameter or local variable shadows a field or property
- Chain constructors — one constructor calling another in the same class
- Pass the current object to another method or return it
- Mark extension method parameters — the this in extension method signatures

Disambiguation — when names collide

```
public class User
{
    private string name;
    private string email;

    public User(string name, string email)
    {
        this.name = name;    // this.name = instance field, name = parameter
        this.email = email;
    }
}

// Modern convention -- _ prefix eliminates need for this:
public class User
{
    private string _name;
    public User(string name) { _name = name; }    // no ambiguity
}
```

Constructor chaining — this(...)

```
public class Order
{
    public int Id { get; }
    public string CustomerName { get; }
    public decimal Amount { get; }
    public string Currency { get; }
    public string Status { get; private set; }
    public DateTime CreatedAt { get; }

    // Primary constructor -- all parameters:
```

```

public Order(int id, string customerName, decimal amount, string currency)
{
    Id = id; CustomerName = customerName;
    Amount = amount; Currency = currency;
    Status = "Pending"; CreatedAt = DateTime.UtcNow;
}

// Convenience -- defaults currency to USD:
public Order(int id, string customerName, decimal amount)
    : this(id, customerName, amount, "USD") { }

// Minimal -- defaults amount too:
public Order(int id, string customerName)
    : this(id, customerName, 0m, "USD") { }
}

```

Execution order: The chained constructor (: this(...)) runs BEFORE the calling constructor's body.

Passing this to external methods

```

public class Order
{
    public void Register(IOrderRegistry registry)
        => registry.Register(this);    // pass current object

    public void Validate(IOrderValidator validator)
    {
        var errors = validator.Validate(this);
        if (errors.Any()) throw new ValidationException(errors);
    }
}

```

Fluent builder pattern — return this

```

public class QueryBuilder
{
    private string _table;
    private List<string> _conditions = new();
    private int? _limit;

    public QueryBuilder From(string table) { _table = table; return this; }
    public QueryBuilder Where(string condition) { _conditions.Add(condition); return this; }
    public QueryBuilder Limit(int count) { _limit = count; return this; }

    public string Build()
    {
        var where = _conditions.Any()
            ? $" WHERE {string.Join(" AND ", _conditions)}" : string.Empty;
        var limit = _limit.HasValue ? $" LIMIT {_limit}" : string.Empty;
        return $"SELECT * FROM {_table}{where}{limit}";
    }
}

var query = new QueryBuilder()
    .From("Orders")

```

```

        .Where("Status = 'Pending'")
        .Where("Amount > 100")
        .Limit(50)
        .Build();

```

this in extension methods

```

public static class OrderExtensions
{
    // 'this Order order' -- extends the Order type:
    public static bool IsHighValue(this Order order) => order.Amount > 10000;
    public static OrderDto ToDto(this Order order)
        => new OrderDto { Id = order.Id, Status = order.Status };
}

var order = new Order { Amount = 15000m };
order.IsHighValue() // true

```

this indexer

```

public class OrderCollection
{
    private readonly Dictionary<int, Order> _orders = new();

    public Order? this[int id]
    {
        get => _orders.TryGetValue(id, out var order) ? order : null;
        set
        {
            if (value is null) _orders.Remove(id);
            else _orders[id] = value;
        }
    }
}

var collection = new OrderCollection();
collection[1] = new Order(); // set via indexer
var order = collection[1]; // get via indexer

```

When you actually need this in modern C#

```

// NEEDED -- constructor chaining:
public Order(int id, string name) : this(id, name, 0m) { }

// NEEDED -- passing current object:
registry.Register(this);
return this; // fluent builder

// NEEDED -- extension method declaration:
public static bool IsValid(this Order order) { }

// NEEDED -- indexer:
public T this[int index] { get => ...; set => ...; }

// OPTIONAL -- disambiguation (use _ prefix convention instead):

```



```

this.name = name;

// NOT NEEDED -- accessing instance members:
this.Status = "Pending";    // just write Status = "Pending"

```

Key Takeaways

- this is a reference to the current instance — the object executing the current method or constructor
- Disambiguation: `this.field = parameter` — distinguishes instance member from local variable
- In modern C# with `_` prefix convention, disambiguation is rarely needed
- Constructor chaining: `: this(args)` — one constructor calls another, chained runs first
- Fluent builders: `return this` — enables method chaining on the same object
- Extension methods: `this T param` — marks the type being extended
- Indexers: `public T this[int index]` — enables `[]` access on custom types

Usage	Syntax	Purpose
Disambiguation	<code>this.field=param</code>	Distinguish field from local var
Constructor chain	<code>: this(args)</code>	Call another constructor
Pass current obj	<code>Method(this)</code>	Send current instance to method
Fluent return	<code>return this</code>	Enable method chaining
Extension method	<code>this T param</code>	Mark extended type
Indexer	<code>public T this[k]</code>	Enable <code>[]</code> access

Memory aid: this is your nametag at a conference. Most of the time you don't need to look at it — everyone knows who you are from context. But when two people have the same name (shadowed field), you point to your nametag (`this.name`) to be clear.

Debugging Tips

Symptom	Cause	Fix
Field always has default value	<code>name = name</code> assigns param self	Add <code>this.</code> or use <code>_</code> convention
Constructor chaining loops	Calls itself instead of overld	Ensure <code>: this()</code> calls differe
Fluent method not chainable	Returns void instead of this	Change return type, add retur
Extension method compile error	<code>this</code> missing from first param	Add <code>this</code> before first param t
Indexer not found	No <code>this[key]</code> implemented	Add <code>public T this[KeyType key]</code>

Self-check — you should now be able to:

- Explain what `this` refers to and in which three situations you genuinely need to write it
- Write constructor chaining with `: this(args)` and explain execution order
- Implement a fluent builder using `return this` for method chaining
- Write an indexer using `this[key]` syntax
- Explain why modern C# code with the `_` prefix convention rarely needs `this` for disambiguation

2.16. Method Hiding vs Overriding (new vs override)

Builds on: 2.6 (inheritance — both hiding and overriding happen in derived classes), 2.7 (polymorphism — overriding enables it, hiding deliberately breaks it), 2.8 (abstract classes — abstract methods require override, never new).

What is the difference and why does it matter?

- **override** replaces the base method. Polymorphism works — calls through a base reference invoke the derived version.
- **new** hides the base method. Polymorphism is broken — calls through a base reference still invoke the base version.

override — polymorphic replacement

```
public class Notification
{
    public virtual void Send() => Console.WriteLine("Notification sent");
}

public class EmailNotification : Notification
{
    public override void Send() => Console.WriteLine("Email sent");
}

Notification n = new EmailNotification();
n.Send();    // "Email sent" -- derived version runs
```

new — method hiding

```
public class EmailNotification : Notification
{
    public new void Send()    // hides, does not override
        => Console.WriteLine("Email sent");
}

Notification n = new EmailNotification();
n.Send();    // "Notification sent" -- BASE version runs!

EmailNotification e = new EmailNotification();
e.Send();    // "Email sent" -- derived version (direct call)
```

Side by side — the exact same setup, different outcomes

```
public class Animal
{
    public virtual string Speak() => "...";
}

// Dog uses override:
public class Dog : Animal
{
    public override string Speak() => "Woof";
}

// Cat uses new:
```

```

public class Cat : Animal
{
    public new string Speak() => "Meow";
}

Animal dog = new Dog();
Animal cat = new Cat();

Console.WriteLine(dog.Speak()); // "Woof" -- override works
Console.WriteLine(cat.Speak()); // "..." -- hiding, base runs!

Cat catDirect = new Cat();
Console.WriteLine(catDirect.Speak()); // "Meow" -- direct call

```

When the compiler warns you

```

public class EmailNotification : Notification
{
    // Warning: hides inherited member. Use new if hiding was intended.
    public void Send() // no override, no new
        => Console.WriteLine("Email sent");
}

```

The code compiles and behaves like new — hiding, not overriding. The warning exists because hiding is almost always accidental.

Accessing the hidden base method

```

Cat cat = new Cat();
cat.Speak(); // "Meow" -- Cat's version

((Animal)cat).Speak(); // "..." -- base version via cast

Animal animalRef = cat;
animalRef.Speak(); // "..." -- base version via base reference

```

When new (hiding) is legitimate

Case 1 — you don't control the base class and it added a method with the same name:

```

// Third-party library added Process() in a new version:
public class YourClass : ThirdPartyBase
{
    public new void Process() // your version still works -- hides library version
        => Console.WriteLine("Your process");
}

```

Case 2 — covariant return types in older C# (C# 9+ supports covariant override now, so new is no longer needed for this).

The realistic bug scenario

```

public abstract class BaseValidator
{
    public virtual bool Validate(object input) { return input != null; }
}

```

```

public class OrderValidator : BaseValidator
{
    // Bug: Developer meant to override but used new:
    public new bool Validate(object input)    // hiding -- not overriding
        => base.Validate(input) && (input is Order o && o.Amount > 0);
}

// In the validation pipeline:
BaseValidator validator = new OrderValidator();
validator.Validate(order);    // OrderValidator's logic NEVER runs!

// Fix:
public class OrderValidator : BaseValidator
{
    public override bool Validate(object input)    // now polymorphism works
        => base.Validate(input) && (input is Order o && o.Amount > 0);
}

```

Key Takeaways

- **override** — replaces the base method. Runtime uses actual object type. Polymorphism works.
- **new** — hides the base method. Runtime uses variable's declared type. Polymorphism breaks.
- The difference only matters when calling through a base class or interface reference
- Omitting both keywords compiles with a warning — behaves like **new**
- **new** is almost always wrong in application code — it's legitimate only when you can't modify the base class
- C# 9+ supports covariant return types in **override** — eliminating the last common use case for **new**

	override	new
-----+-----+-----		
Base must	virtual or abstract	Any (virtual or not)
Dispatch	Actual object type	Declared variable type
Polymorphism	Works correctly	Broken
Warning	Yes if omitted	Silenced by adding new
Use when	Almost always	Can't modify base class

Memory aid: **override** = upgrade the engine in the same car body. Everyone who drives 'a car' gets the new engine. **new** = build a different car with the same name. People asking for 'a car' still get the old model — only people specifically asking for your exact brand get the new one.

Debugging Tips

Symptom	Cause	Fix
-----+-----+-----		
Derived method never called	Uses new instead of override	Change new to override
Compiler warning: hides member	Missing new keyword	Add override (preferred) or new
Cannot use override	Base method not virtual	Add virtual to base method
Mock doesn't intercept calls	Method not virtual	Make method virtual or abstract

Self-check — you should now be able to:

- Explain the runtime difference between `override` and `new` with a concrete code example
- Demonstrate that `override` respects the actual object type while `new` respects the declared variable type
- Explain when a compiler warning about method hiding means you should use `override` instead of `new`
- Identify the narrow legitimate cases where `new` is appropriate
- Debug a scenario where a derived class method is never called because it uses `new` instead of `override`

2.17. Object Equality — `Equals`, `GetHashCode`, `==` Overloading

Builds on: 2.1 (classes — equality is defined on classes), 2.13 (records — records auto-generate correct equality), 1.4 (value vs reference types — equality semantics differ between them), 2.7 (polymorphism — `Equals` is a virtual method on object).

What is equality in C# and why is it complex?

In C#, equality isn't one thing — it's a contract between three mechanisms that must stay consistent: `Equals(object obj)`, `GetHashCode()`, and the `==` operator.

By default, classes use reference equality — two variables are equal only if they point to the same object in memory. This is usually wrong for domain objects and DTOs.

```
public class OrderId
{
    public int Value { get; }
    public OrderId(int value) => Value = value;
}

var a = new OrderId(42);
var b = new OrderId(42);

Console.WriteLine(a == b);           // false -- different objects
Console.WriteLine(a.Equals(b));      // false -- different objects
```

The equality contract — four rules that must hold

- Reflexive: `x.Equals(x)` must return true
- Symmetric: if `x.Equals(y)` is true, then `y.Equals(x)` must be true
- Transitive: if `x.Equals(y)` and `y.Equals(z)`, then `x.Equals(z)` must be true
- Consistent with `GetHashCode`: if `x.Equals(y)` is true, `x.GetHashCode() == y.GetHashCode()` must be true

Breaking rule 4 causes silent, catastrophic bugs in dictionaries and hash sets.

Overriding `Equals`

```
public class OrderId
{
```

```

public int Value { get; }
public OrderId(int value) => Value = value;

public override bool Equals(object? obj)
{
    if (ReferenceEquals(this, obj)) return true;
    if (obj is not OrderId other) return false;
    return Value == other.Value;
}

public override int GetHashCode() => Value.GetHashCode();
}

Console.WriteLine(new OrderId(42).Equals(new OrderId(42))); // true
Console.WriteLine(new OrderId(42).Equals(null));           // false

```

Overriding GetHashCode — the critical partner

Golden rule: if two objects are equal (Equals returns true), they **MUST** have the same hash code.

```

// Broken -- Equals overridden but GetHashCode not:
public class OrderId
{
    public int Value { get; }
    public override bool Equals(object? obj)
        => obj is OrderId other && Value == other.Value;
    // GetHashCode NOT overridden -- reference-based hash!
}

var dict = new Dictionary<OrderId, string>();
var id1 = new OrderId(42);
dict[id1] = "Order A";

var id2 = new OrderId(42); // equal to id1 by Equals
dict.ContainsKey(id2);     // FALSE -- different hash code!
dict[id2];                 // KeyNotFoundException -- catastrophic bug

```

Using GetHashCode.Combine — the modern approach:

```

// Single field:
public override int GetHashCode() => Value.GetHashCode();

// Multiple fields:
public override int GetHashCode()
    => GetHashCode.Combine(FirstName, LastName, DateOfBirth);

```

Implementing IEquatable

```

public class OrderId : IEquatable<OrderId>
{
    public int Value { get; }
    public OrderId(int value) => Value = value;

    // Typed equality -- no boxing, no casting:
    public bool Equals(OrderId? other)
    {

```

```

        if (other is null) return false;
        if (ReferenceEquals(this, other)) return true;
        return Value == other.Value;
    }

    // object.Equals delegates to typed version:
    public override bool Equals(object? obj) => Equals(obj as OrderId);

    public override int GetHashCode() => Value.GetHashCode();
}

```

Overloading == and !=

```

public class OrderId : IEquatable<OrderId>
{
    // ... Equals and GetHashCode as above ...

    // Overload == and != together -- always do both:
    public static bool operator ==(OrderId? left, OrderId? right)
    {
        if (left is null && right is null) return true;
        if (left is null || right is null) return false;
        return left.Equals(right);
    }

    public static bool operator !=(OrderId? left, OrderId? right)
        => !(left == right);
}

var a = new OrderId(42);
var b = new OrderId(42);
Console.WriteLine(a == b);    // true
Console.WriteLine(a != b);    // false
Console.WriteLine(a.Equals(b)); // true -- all consistent

```

Records — equality is already done correctly

```

public record OrderId(int Value);

var a = new OrderId(42);
var b = new OrderId(42);

Console.WriteLine(a == b);    // true -- generated
Console.WriteLine(a.Equals(b)); // true -- generated
Console.WriteLine(a.GetHashCode() == b.GetHashCode()); // true -- generated

```

If your type is primarily a data carrier with no mutable state, use a record instead of manually implementing equality. The generated implementation is correct, optimised, and consistent.

Full implementation — complete example

```

public sealed class Money : IEquatable<Money>
{
    public decimal Amount { get; }
    public string Currency { get; }
}

```

```

public Money(decimal amount, string currency)
{
    if (amount < 0) throw new ArgumentException("Amount cannot be negative");
    Amount = amount;
    Currency = currency?.ToUpperInvariant()
        ?? throw new ArgumentNullException(nameof(currency));
}

public bool Equals(Money? other)
{
    if (other is null) return false;
    if (ReferenceEquals(this, other)) return true;
    return Amount == other.Amount && Currency == other.Currency;
}

public override bool Equals(object? obj) => Equals(obj as Money);

public override int GetHashCode() => HashCode.Combine(Amount, Currency);

public static bool operator ==(Money? left, Money? right)
{
    if (left is null && right is null) return true;
    if (left is null || right is null) return false;
    return left.Equals(right);
}

public static bool operator !=(Money? left, Money? right) => !(left == right);

public override string ToString() => $"{Amount:F2} {Currency}";
}

var price1 = new Money(99.99m, "USD");
var price2 = new Money(99.99m, "usd"); // normalised to USD
Console.WriteLine(price1 == price2); // true

var prices = new HashSet<Money> { price1, price2 };
Console.WriteLine(prices.Count); // 1 -- duplicates removed correctly

```

Key Takeaways

- Default class equality is reference-based — two objects with identical data are not equal
- Overriding Equals without overriding GetHashCode causes catastrophic bugs in collections
- If `x.Equals(y)` is true, `x.GetHashCode()` must equal `y.GetHashCode()` — this is non-negotiable
- Overriding Equals does NOT change `==` — you must overload `==` separately
- Always overload `==` and `!=` together
- Implement `IEquatable` alongside `Equals(object)` for typed, non-boxing equality
- Use records for value objects — equality is generated correctly and automatically

Step	Method	Required?
1	override <code>bool Equals()</code>	Yes
2	override <code>int GetHashCode</code>	Yes -- always with Equals

3	bool Equals(T?) IEquat	Yes -- strongly recommended
4	operator ==(T?, T?)	Yes if == should be struct
5	operator !=(T?, T?)	Yes -- always with ==

.NET-specific rules:

- `HashCode.Combine(field1, field2, ...)` — use this, not manual XOR
- `ReferenceEquals` — use when you need reference comparison after overriding `==`
- Records auto-generate all five — prefer records for value objects
- EF Core change tracking depends on correct `GetHashCode` — wrong hash codes cause missed updates

Memory aid: `Equals` and `GetHashCode` are partners — they must agree. If `Equals` says two objects are twins, `GetHashCode` must give them the same fingerprint. A twin with a different fingerprint will get lost in any database (dictionary, `HashSet`) that uses fingerprints to find people.

Debugging Tips

Symptom	Cause	Fix
Dict.ContainsKey returns false	GetHashCode inconsistency	Override GetHashCode
HashSet contains duplicates	Same cause	Override GetHashCode
a.Equals(b) true but a==b false	== not overloaded	Overload == and !=
LINQ Distinct doesn't remove dups	Missing Equals/GetHashCode	Implement IEquatable<T>
EF Core misses entity modification	GetHashCode is unstable	Base hash on immutable field

.NET-specific: `GetHashCode` must be stable for the lifetime of an object when it's in a collection.

Never compute the hash from mutable fields that might change while the object is in a hash-based collection.

```
// DANGEROUS -- mutable field in GetHashCode:
public class Order
{
    public string Status { get; set; } // mutable
    public override int GetHashCode() => Status.GetHashCode(); // changes!
}

var set = new HashSet<Order>();
var order = new Order { Status = "Pending" };
set.Add(order);
order.Status = "Shipped"; // hash changed -- order is lost!
set.Contains(order); // false!
```

Self-check — you should now be able to:

- Explain why overriding `Equals` without `GetHashCode` is dangerous and demonstrate the bug
- Implement the complete equality pattern (`Equals`, `GetHashCode`, `IEquatable`, `==`, `!=`) for a class
- Explain why `==` and `Equals` are separate and when each needs to be overridden
- Use `HashCode.Combine` to produce correct hash codes from multiple fields
- Explain why records are preferred for value objects — what they auto-generate and why it matters

- Identify why mutable fields should not be used in GetHashCode

Section 2 complete. All 17 topics from 2.1 through 2.17 are covered. Say next to begin Section 3: Generics.